Week 1

Transformer Architecture; Generative AI Project Lifecycle

Generative AI is a subset of Machine Learning. The models that underpin Generative AI learned these abilities by finding statistical patterns in massive datasets that were originally generated by humans.

They were trained with extremely large datasets, for long times with lots of parameters.

Foundation Models: **GPT, LLaMa, BLOOM, BERT, FLAN-T5, PaLM**

Large Language models take natural language or human written tasks(prompts) and perform those tasks, much as a human would.

Prompt → text that you pass to LLM
Context Window → Memory or space that is available to the LLM.
Completion → output of the model
Inference → using the model to generate text

LLM Use cases and tasks

- Summarizing
    - Summarizing a given text
- Inferring
    - Inferring info, based on a given text
- Extraction
    - Extract specific info, based on given text
- Translation
    - Converting, or translating a given text to another language/context
- Expansion
    - Generate new text, based on prompt and given text

Text generation before transformers

Generating text with RNNs → limited by amount of compute and memory to perform well at generative tasks.

With just one previous word seen in any model, prediction cannot be that good.

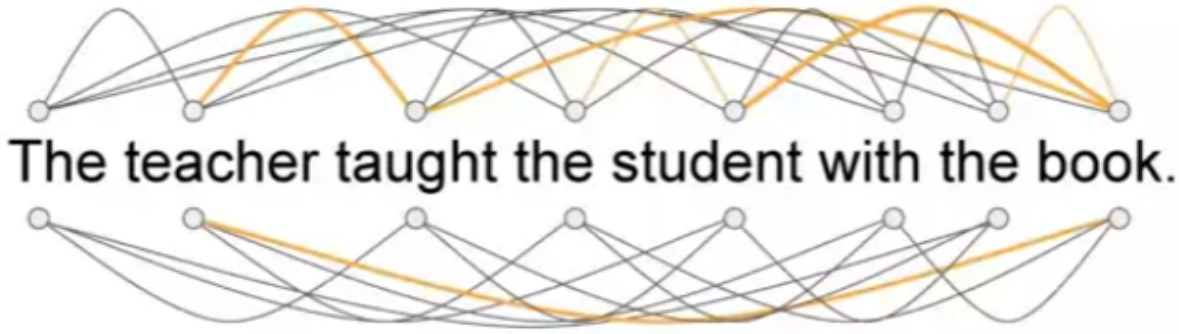
Models need to have understanding, or “context”.

Transformers, solve this problem

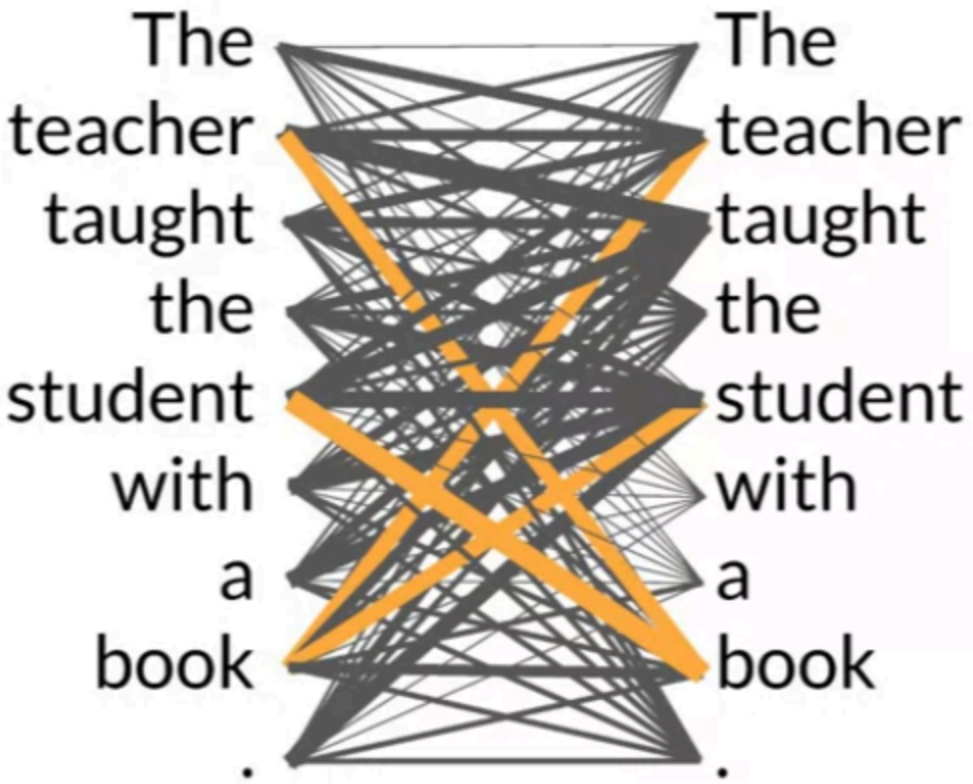
- They scale efficiently to use multi core gpus
- It can parallel process input data
- Pays attention to meaning of the words it's provided

Transformers Architecture

- Power lies in its ability to learn the relevance and understand context of each word in the sentence.
- Also not just to each word next to each other, but each word to each word

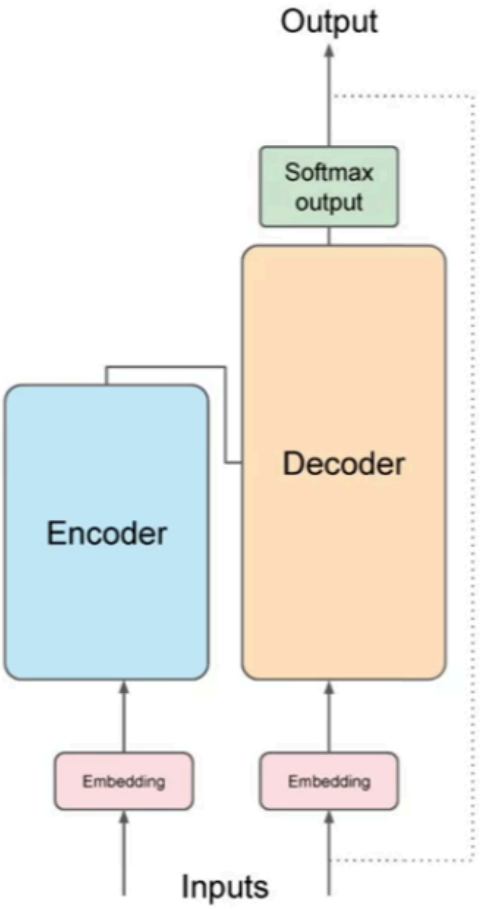


- The LLM applies attention weights to those relationships, so the model can learn the relevance of each word, with each other word.
    - This gives the algorithm the ability to understand context to a deeper level.
    - These attention weights are learned during LLM training.
    - Attention Map

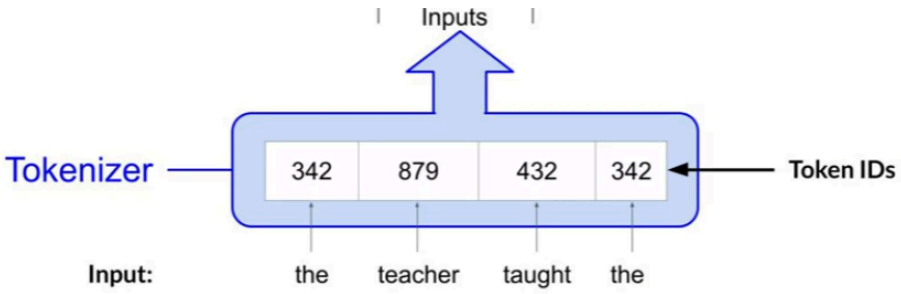


-
- As you can see here, the word book is heavily connected to the words, student, and teacher.

    - Self Attention: The ability to learn attention across a sentence
- Transformer Architecture
    - Simplified Diagram

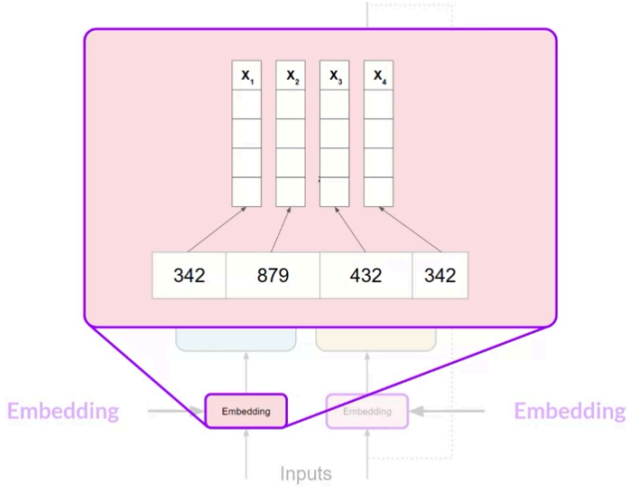


- Before passing an input into the transformer, you must first tokenize each of the words in the input. As networks are large statistical models, they work with numbers.

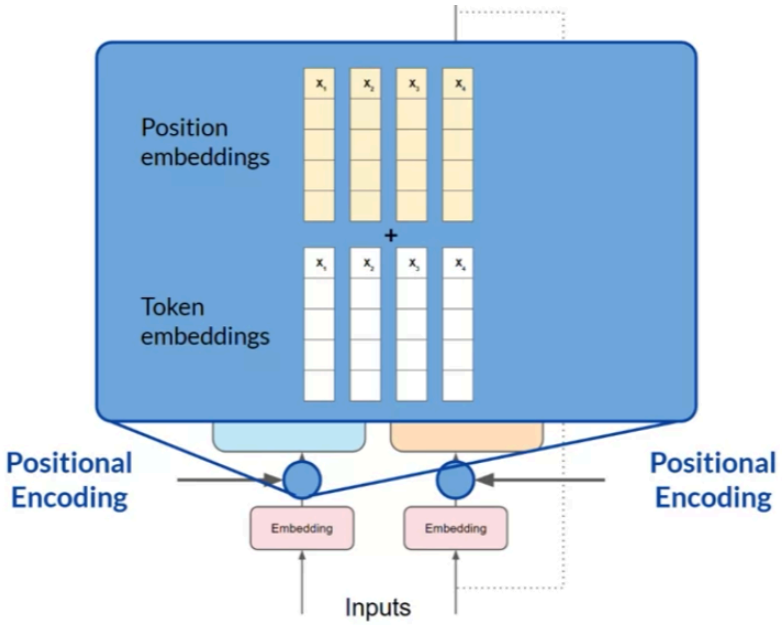


  - Important note: the tokenizer you use to tokenize input, you also must use when generating the text in the output, as the LLM will also just output numbers according to the original encoding.
- After you process your inputs and tokenize them with a given tokenizer, you pass them to the embedding layer. This layer is a trainable vector embedding space, a high dimensional space, where each token is represented as a vector, and occupies a unique location in the space.

        - Each token id in vocabulary is mapped to a multidimensional vector, with the intuition that these vectors learn to encode the meaning and context of individual tokens in the input sequence.

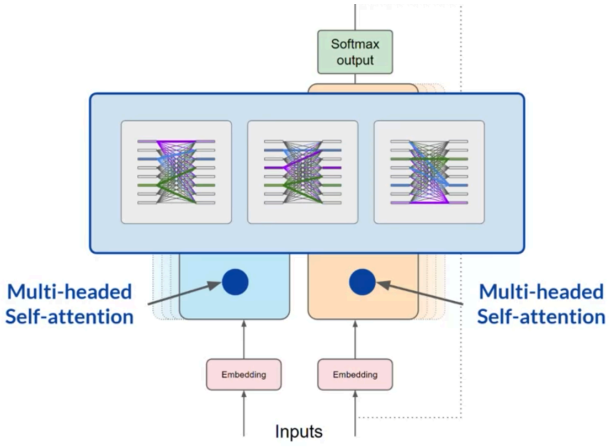


        -
    - As you add the token vectors into the base of the encoder or the decoder, you also add positional encoding. The idea is that the model processes each of the input tokens in parallel, so keeping the original positions of the words would be difficult
    - Positional encodings solve this problem, it preserves the information about the word order. You don't lose the relevance of the position of the word in the sentence.

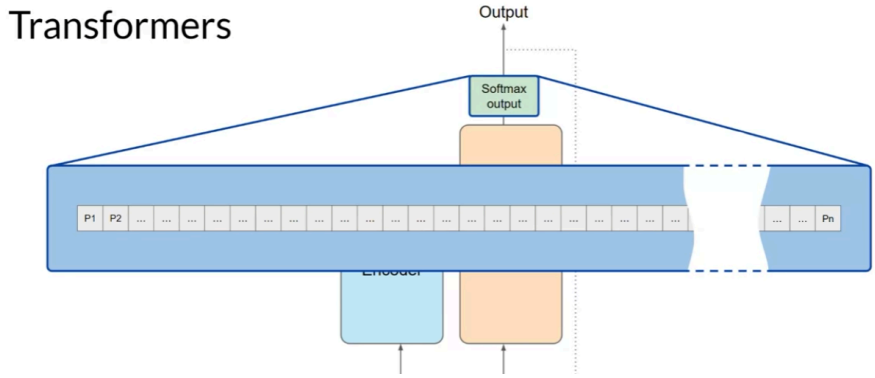


    -
    - Once summing the input tokens, and the positional encodings, you pass the resulting vectors to the self-attention layers. Here the model analyzes the

relationships between the tokens in your input sequence. Here is where it better understands the contextual dependencies between the words.

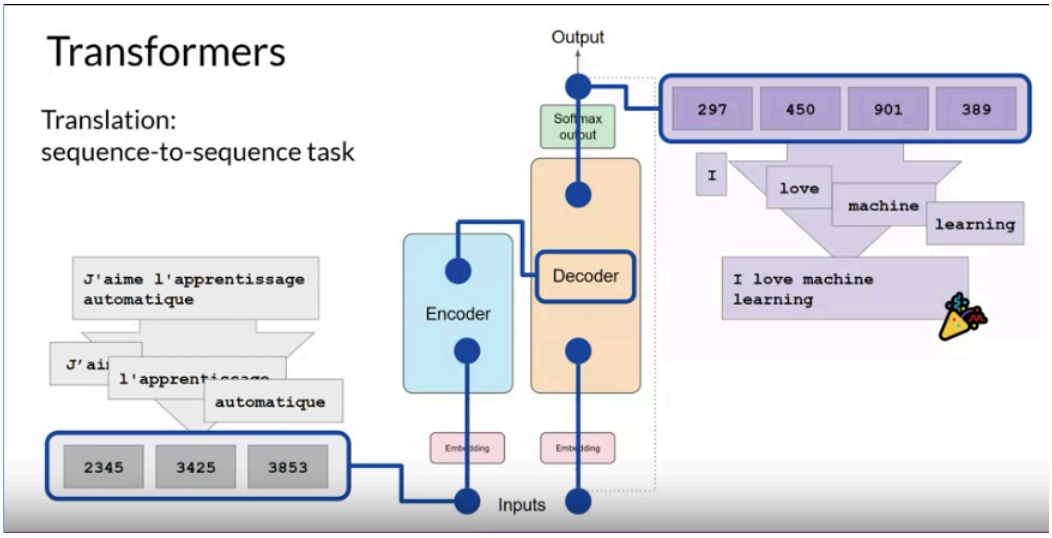
- The self-attention weights that are learned during training, and stored within the layers, are the importance of each word in that sequence to all other words in the sequence.
  - However, the transformer architecture has multi-headed self-attention, this means that multiple sets of self-attention weights are learned in parallel. The number of self-attention heads vary from model to model, however range from 12-100 are common.
  - Each attention head will learn a different aspect of language
    - One headmay learn about the actions in the sentence
    - One head may learn about the entities in the sentence
    - One head may even learn if words rhyme or not, this is the power of attention!
    - However, it is important to note that you do not dictate the aspect of language the specific head is to learn ahead of time, the weights of each head are randomly initialized and given sufficient training data and time, each will learn different aspects of language.
      - While some attention maps are easy to interpret, others may not be
  - Multi Headed self attention:
    - 

- Now that the self-attention weights have been applied to your input data, the output is processed through a fully connected feed forward network. The output of this layer is a vector of logits proportional to the probability score for each and every token in the tokenizer dictionary. You can then pass these logits to a final softmax layer, where they are normalized into a probability score for each word.
  - The output contains a probability for every single word in the vocabulary

        - One single token will have the highest predicted token, but there are a number of methods you can use to vary the selection from this vector of probabilities.
    - 

Generating text with transformers

- Overall end to end prediction process for transformers
- Translation: sequence to sequence task
- Example: Translate “J’aime l'apprentissage automatique”
    - First, tokenize input using same tokenizer used to train the network
    - These tokenized inputs, are then passed to the encoder side of the network
        - Fed into the embedded layer, and then passed on to the multi-headed attention layers
        - The output of the multi headed attention layers, are then passed into a feed forward network to the output of the encoder,
        - At this point, the output that leaves the encoder is a deep representation of the structure and meaning of the input sequence.
    - This representation is then passed on to the decoder
        - The representation/output is then inserted into the middle of the decoder to influence the decoders self-attention mechanisms
        - A “start of sequence” token is added to the input of the decoder, this triggers a prediction to the next token, which is based on the contextual understanding derived from the encoder.
        - The output of the decoders self-attention layers is then passed through to the decoders feed forward network, and through a final softmax output layer,
        - At this point, we have our first token
    - We continue this loop, passing the output token back to the input to trigger the next output token prediction, until the model predicts an end of sequence token
    - At the end, each token can be de-tokenized to words, and you have your output.
- Visual Representation

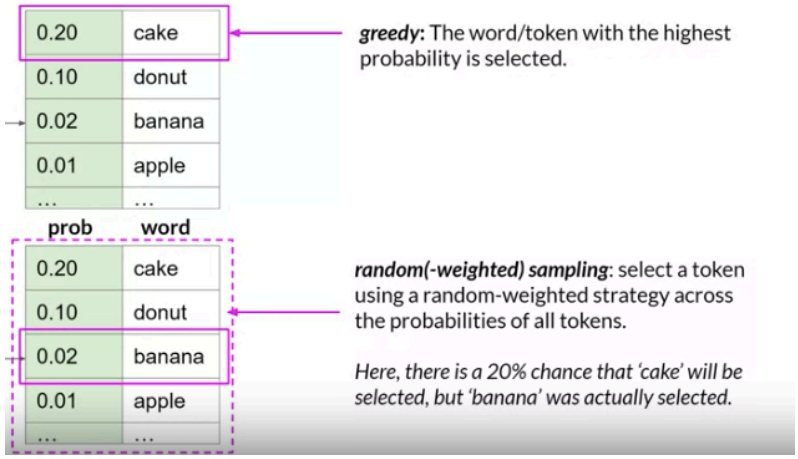
- 

- While translation models use both encoder and decoder components, some models need only some components, not both
    - Encoder only models, also work as sequence to sequence models, you can train these models to do classification processes, such as sentiment analysis
    - Decoder only models, most commonly used today. Can generalize to do most tasks, GPT models use this structure.

Prompting and Prompt Engineering

- **Prompt:** Text you feed into the model
- **Inference**: Act of generating text
- **Completion:** output text from a given prompt + inference
- **Context Window:** Full amount of text or the memory that is available to use from the prompt
- Oftentimes, the completion from a given model is not what's desired, so we have to modify or adjust the prompt we provide → prompt engineering
- **Prompt Engineering:** Act of adjusting, refining, optimizing prompts for LLMs.
    - **In-Context Learning**: Providing examples within a prompt (I believe is also called few shot prompting)
        - No examples → Zero-shot inference/prompting
        - Smaller models perform better with few shot prompting, and may require this for learning, however larger models are quite good with zero-shot prompting

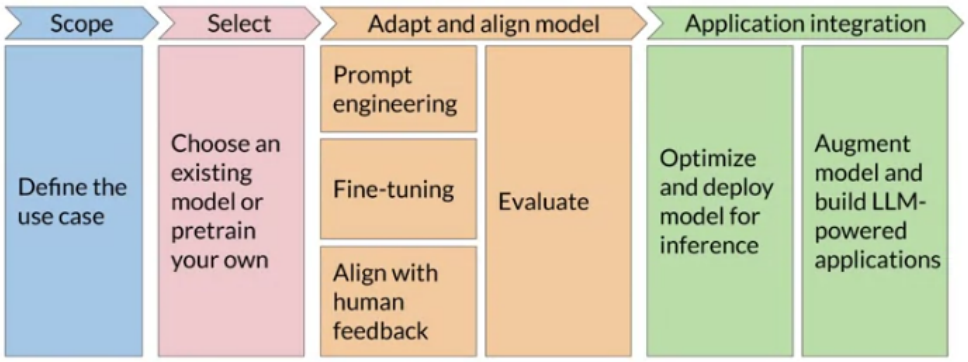
Generative Configuration

- Examine methods and configuration parameters that we can use to influence the way models make decisions
- **Max new tokens**: maximum number of tokens the model can put in its predictions
- Greedy vs. Random Sampling
    - Greedy Decoding: Always choose the word with the highest probability
        - Simplest form of next word prediction
        - Good for short generation
        - Susceptible to repeated words, or repeated sequences of words
    - Random Sampling: Instead of choosing most probable word every time, with random sampling, the model chooses an output word at random using a probability distribution to weight the selection
        - Better for generating natural language
        - There is a chance this method will be "too creative" when the model decides to select words that are going to far in other directions
    - Illustration:
        - 

    - Sampling Techniques
        - Top K
            - Select an output from the top k results, after applying random-weighted strategy using the probabilities
        - Top P
            - Select an output using the random weighted strategy with the top-ranked consecutive results by probability and with a cumulative probability <= p
        - Temperature
            - Higher the temperature, higher the randomness, lower, etc
            - Alters predictions the model makes
            - Cooler temperature (e.g. < 1) strongly peaked distribution, clear answer
                - Less random

        - Similar to greedy
    - Higher temperature (e.g. > 1) broader flatter distribution

Generative AI Project Lifecycle

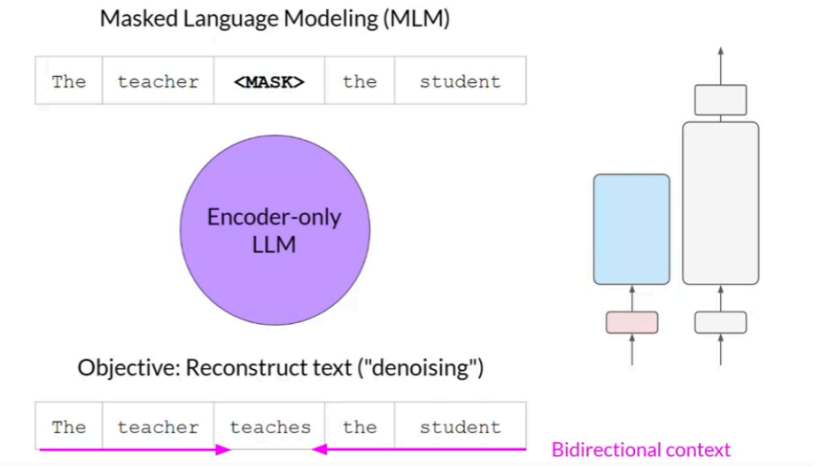
- Diagram:
    - 

- Step 1: Scope
    - **Define the use case**
    - Accurately and as narrowly as you can
    - Abilities of LLMs depend greatly on size and architecture of the model
        - Do you need the LLM to be good at many tasks? Or some smaller ones?
        - Or smaller easier tasks, such as named entity recognition
        - Be sure you have scoped your models needs and abilities enough to go on to the next task
- Step 2: Choose an existing model or pre-train your own
    - In general, you will start with an existing model, although some cases to train a model from scratch
- Step 3: Adapt and Align Model
    - Prompt Engineering
        - Can sometimes be enough for your model to perform well
        - Start with this
            - In context learning
    - Fine-Tuning
        - Supervised Learning Process
    - Align with human feedback
        - RLHF: Help to make sure model behavior is correct
    - Evaluation
        - How well aligned your model is to your preferences
        - Highly iterative process
            - Start with Prompt Engineering

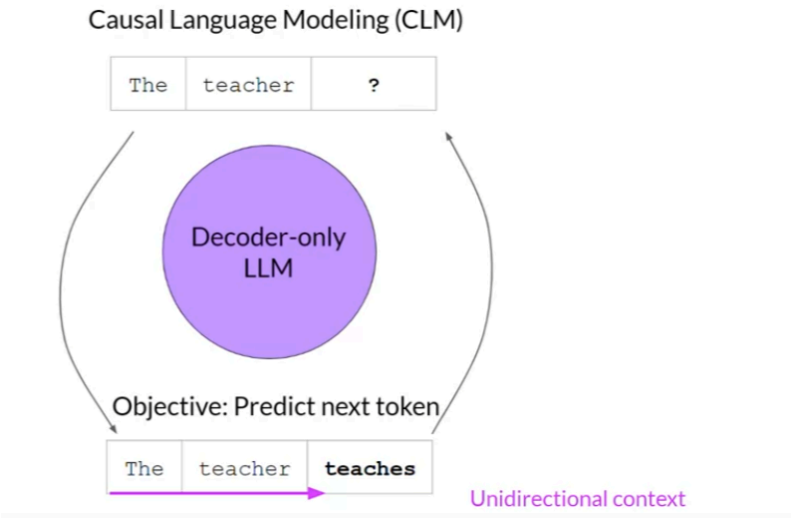
- Step 4: Application Integration
    - Optimize and Deploy model for inference
    - Augment model and build LLM-powered applications

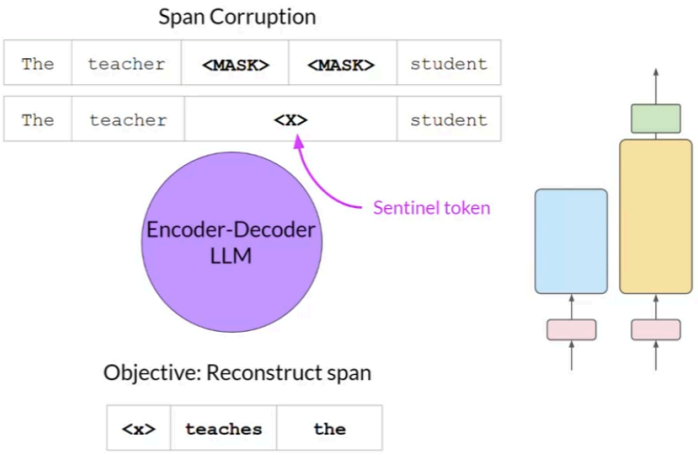
Pre-Training large language models

- You have two choices with your model, choose a pre-trained from foundation model, or train your own custom LLM
    - Some specific circumstances where training your own model from scratch is beneficial
    - In general, a foundation model is the better choice
- Major frameworks, such as Pytorch and HuggingFace have hubs that have foundation models that you can browse
    - Model Cards: describe important details on models, such as use cases, how it was trained, advantages and disadvantages.
- Model architectures and pre-training objectives
    - Pre-training phase
        - As learned earlier, LLMs encode a deep statistical understanding of language
        - Model learns from vast amounts of textual data
            - Up to petabytes of data
            - Data pulled from multiple sources
        - In this step, model internalizes the patterns and structures of the language
            - Allows model to complete its training objective
                - Depends on architecture of the model
            - During training, the model updates its weights to minimize the training loss
            - The encoder generates an embedding representation for each token
            - Overall, this requires a large amount of compute, and the need for gpus
        - Important Note: When scraping data from the web, you often have to pre process the data to increase quality, remove bias and noise. As a result, only 1-3% of data is used during pre-training
    - Encoder only models
        - Autoencoding models
        - Pre-trained using masked language modeling (MLM)
            - Tokens in the input sequence are randomly masked, and the training objective is to reconstruct the text
                - Also called denoising objective

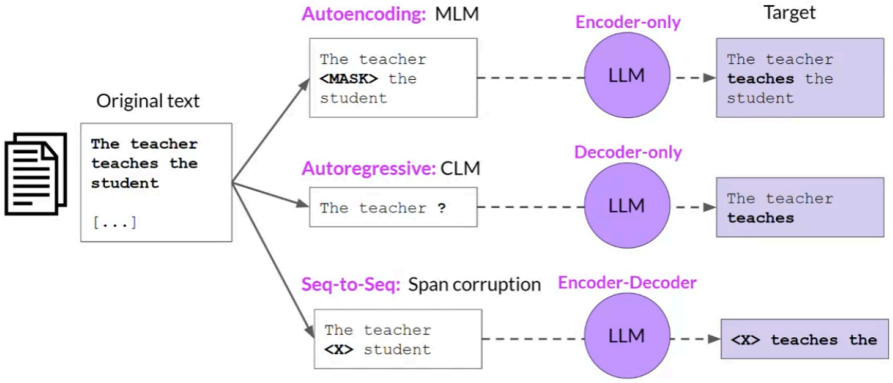
- This type of model also build bi-directional representations of the input sequence, allowing for full contextual understanding
- These models are ideally suited for these problems
  - Sentiment Analysis
  - Named entity recognition
  - Word Classification
- Diagram
- 

- Decoder only models
  - Autoregressive models
    - Pre-trained using causal language modeling
      - Predict next token based on previous sequence of tokens
      - Mask input sequence, and can only see input tokens leading up to token in question
      - Context is unidirectional
    - Good use cases
      - Text generation
      - Larger models, do show high potential for zero-shot inference capabilities
      - GPT is a Autoregressive model
    - Diagram

- 

- Encoder-Decoder models
  - Sequence to sequence models
  - Pre-trained using Span Corruption
    - Masks random sequences of input tokens
    - Masked sequences are then replaced with a sentinel token
      - Sentinel tokens are tokens added to the vocabulary that do not correspond with any actual word on the input text
    - Decoder reconstructs the corrupted span autoregressively
  - Good use cases
    - Translation
    - Text Summarization
    - Question answering
  - Diagram
    - 

- Overall Comparison



- Important to note: The training objective sometimes varies from model to model.
- Larger models of any architecture are more capable of carrying out the task well.
- Larger the model → better performance

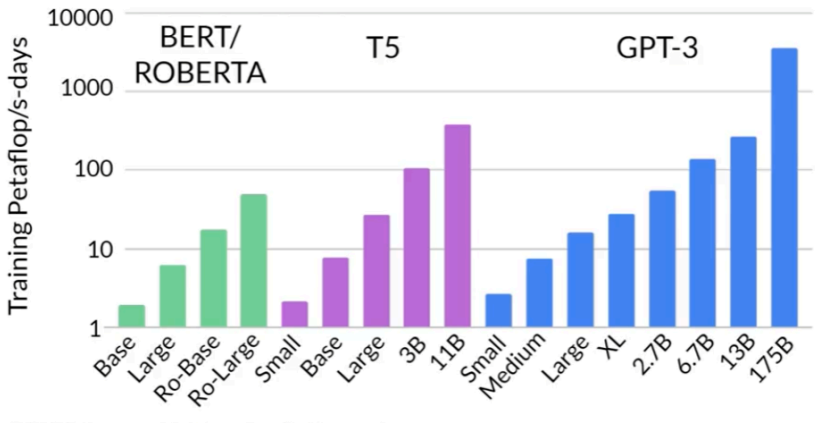
Computational Challenges of Training LLMs

- OutOfMemoryError: CUDA out of memory
  - Compute unified device architecture
- Libraries like pytorch, use CUDA to boost performance on matrix multiplication and other operations within deep learning
- To train a 4GB 32 bit precision model, you need 24GB 32-bit precision, around6 times the memory
- To reduce memory we can use:
  - Quantization
    - Reduce precision of weights to 16-bit floating point numbers, rather than 32-bit floating point numbers
    - BFLOAT16 is the popular choice for quantization

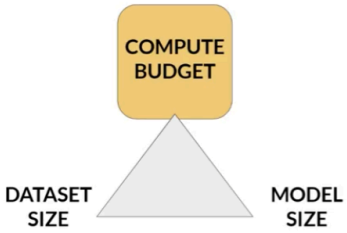
Scaling laws and compute-optimal models

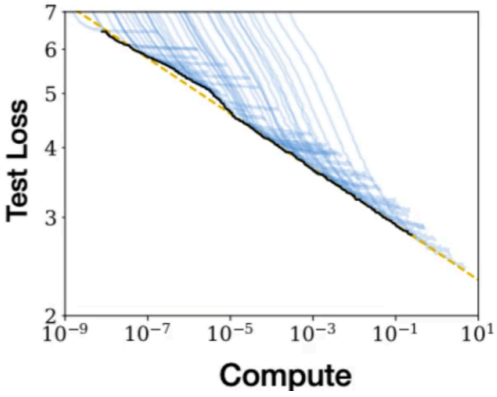
- Scaling choices for pre-training
  - Goal: maximize model performance of learning objective
    - I.e minimize loss
  - 2 choices
    - Increase size of dataset
    - Increase model complexity (number of parameters)
  - These choices are constrained by the compute budget, i.e GPUs, training time, and cost
- Units for compute, that quantifies resources

    - petaflop/s-day = floating point operations performed at a rate of 1 petaFLOP per second for one day
    - 1 petaFLOP/s = 1 000 000 000 000 000 floating point operations per second
- Visualize the scale



- Huge amount of compute for larger models
    - Compute budget vs. model performance





Pre-training for domain adaptation

- Legal Language
    - Makes use of very specific terms
    - Models may have difficulty understanding specific terms, as these aren't used daily in the natural language.
        - With this, some words used in the natural language are used differently, and this also may skew the performance of the model
- Pre-training from scratch, will help for highly specialized domains that have these issues.
    - BloombergGPT

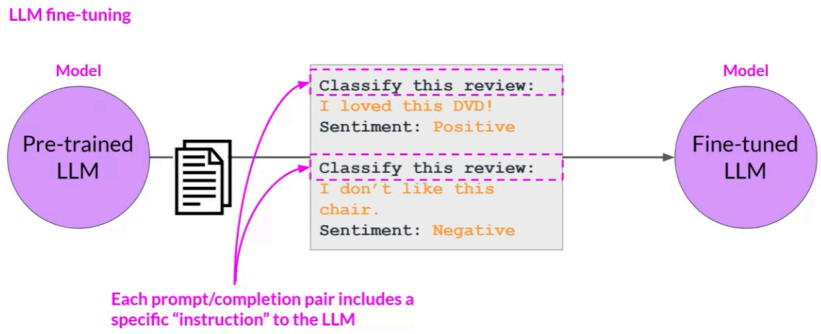
Week 2

Fine Tuning and evaluating large language models

Instruction fine tuning

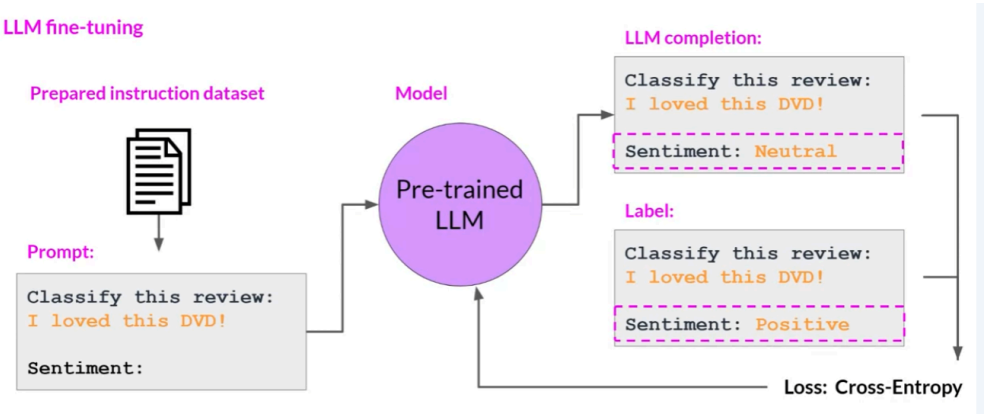
Fine tuning an LLM with instruction prompts

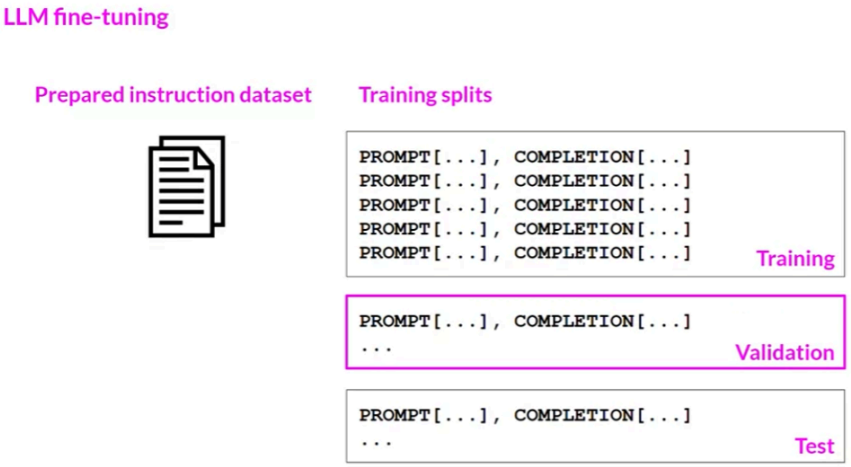
- In context learning (ICL)
    - Zero shot learning
    - Few shot learning
    - Drawbacks
        - May not work with smaller models, even with 5 or 6 examples
        - Examples take up token space within the context window
- Fine-tuning
    - Pre-training → vast amounts of unstructured text data via self supervised learning (RL-esque)
    - **Fine-tuning**: supervised learning process where a dataset of labeled examples is used to update the weights of an LLM
        - Labeled examples: prompt-completion pairs
    - This extends the models ability to do good completions of a prompt from a given domain
    - **Instruction Fine Tuning**:
        - Trains model using examples that demonstrate how it should behave to certain prompts



        - The dataset itself however (showed above as the black box) will have many pairs of prompt completion examples for the task you are interested in
        - **Full fine-tuning**: when all parameters(weights) are updated
        - This all results in a new improved performance model that is fine tuned to a specific domain

- Full fine-tuning also requires a large compute and memory.
- Use prompt templates to help design your datasets (langchain)
- Once you have the dataset, you separate your data, into test, training, and validation sets
- Using these, and depending on the task that is being fine tuned on, calculate the loss by comparing responses of your current model, and the label.
- Use this loss to update your model weights (standard backpropagation) ; do this over many batches of prompt-completion pairs, and over many epochs, so that the model's performance on the task improves.





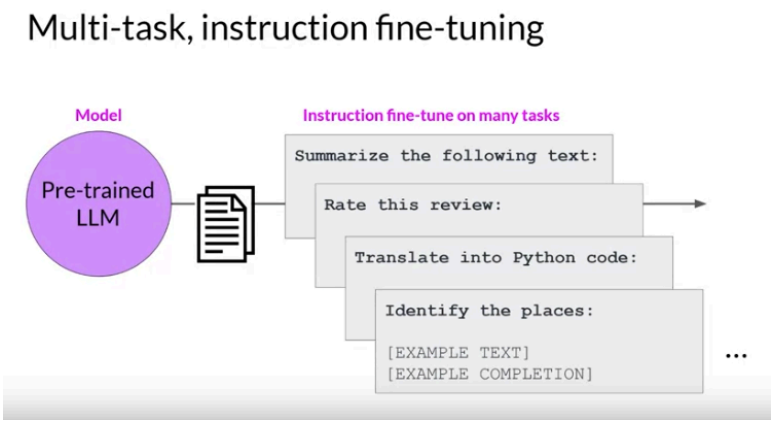
Fine tuning on a single task

- Single-task training
  - E.g. summarization
- Often only need 500-1000 examples to fine-tune a single task
- Potential Downside of training on a single task:
  - **Catastrophic Forgetting**

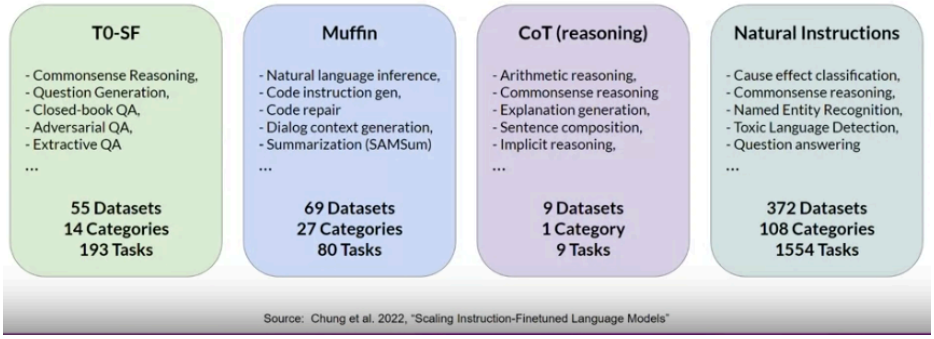
        - Fine-tuning can significantly increase the performance of a model on a specific task
        - However, catastrophic forgetting occurs due to fine-tuning altering the weights of the original model
            - This leads to great performance on the original fine tuning task, however will degrade performance on other tasks
        - To avoid catastrophic forgetting
            - First, decide whether it is a real issue or not
                - If all you need is reliable performance on the current task, you do not need to worry about catastrophic forgetting
                - If you need to maintain the multi-task capabilities of the original model:
                    - Fine tune on multiple tasks at the same time
                        - Good multi task fine tuning, may require 50000-100000 examples across many tasks
                        - Lots of data and compute
                    - **Parameter-Efficient Fine Tuning (PEFT)**
                        - Set of techniques that preserves weights of the original LLM and trains only a small number of task specific adapter layers and parameters
                        - Shows greater robustness to catastrophic forgetting since most of the pre-trained weights are left untrained

Multi-Task Fine Tuning

- An extension of single task fine tuning, where training dataset is comprised of example input and outputs on a number of tasks, rather than one



    - Drawback: needs lots of data
- Instruction Fine-Tuning with FLAN (Fine tuned language net)

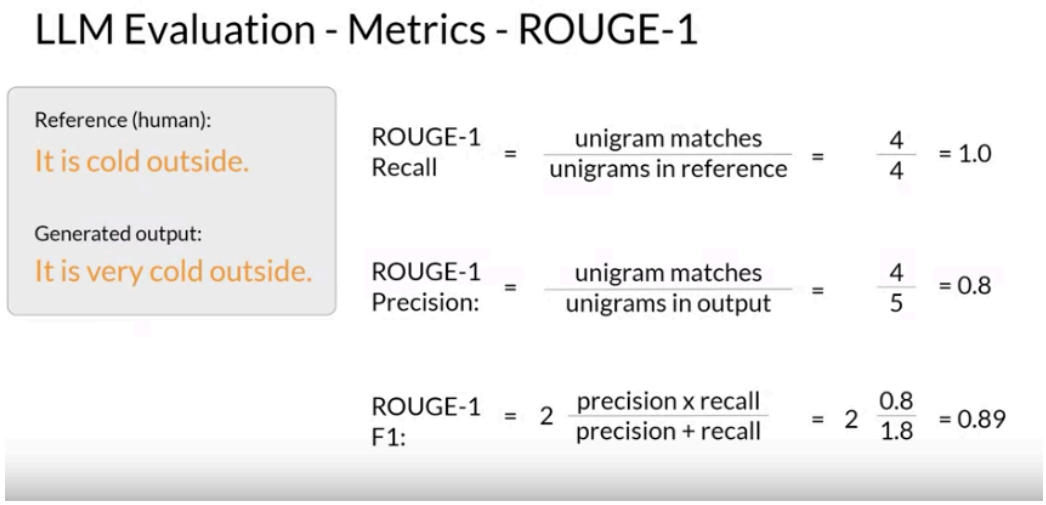
- FLAN models refer to a specific set of instructions used to perform instruction fine-tuning
    - FLAN-T5
        - FLAN instructed model of base T5 model
        - Great general purpose model
        - 

    - FLAN-PALM
        - FLAN instructed model of base PALM model
- In fine tuning, some of the tasks have prompts templates to aid the model in training
    - I.e, different ways of saying summarize this dialogue
        -
```
"samsum": [
       ("{dialogue}\n\Briefly summarize that dialogue.", "{summary}"),
       ("Here is a dialogue:\n{dialogue}\n\nWrite a short summary!",
        "{summary}"),
       ("Dialogue:\n{dialogue}\n\nWhat is a summary of this dialogue?",
        "{summary}"),
       ("{dialogue}\n\nWhat was that dialogue about, in two sentences or less?",
        "{summary}"),
       ("Here is a dialogue:\n{dialogue}\n\nWhat were they talking about?",
        "{summary}"),
       ("Dialogue:\n{dialogue}\nWhat were the main points in that "
        "conversation?", "{summary}"),
       ("Dialogue:\n{dialogue}\nWhat was going on in that conversation?",
        "{summary}"),
]
```
    - You should fine-tune with your own personal dataset, within your own company, to get the best instruct model

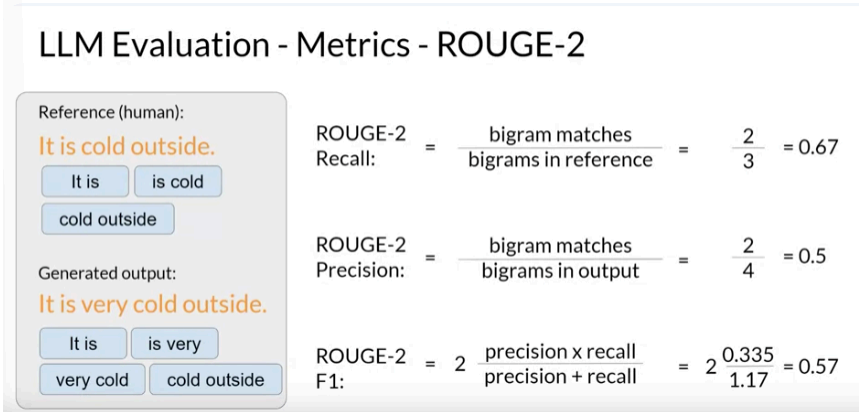
Model Evaluation

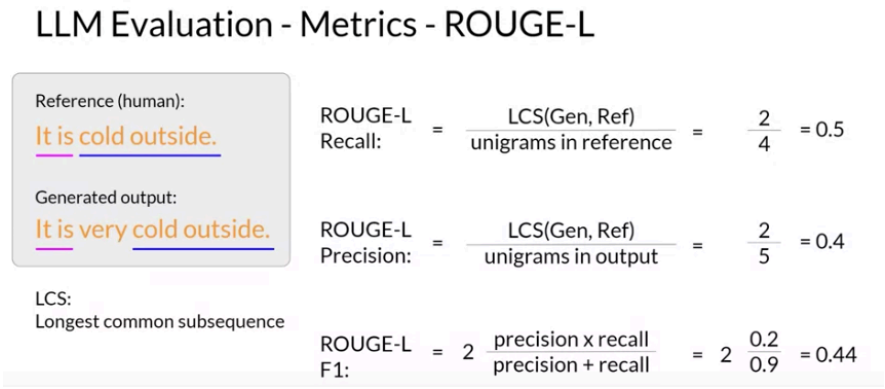
- Model Evaluation Metrics

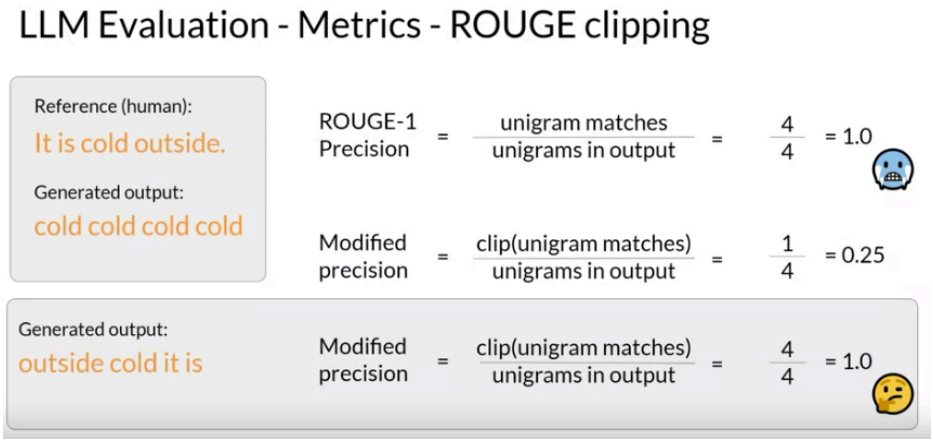
- In traditional ML models, you look at the performance on training and validation datasets, where the output is already known
  - Simple metrics such as accuracy, as most traditional ML models are deterministic
- ROUGE
  - Recall Oriented Understudy For Gisting Evaluation
    - Primarily employed to assess the quality of automatically generated summaries, by comparing them to human generated reference summaries
- BLEU SCORE
  - Bilingual Evaluation Understudy
    - Algorithm designed to evaluate the quality of machine translated text, in comparison to human translated text
    - Sometimes called “Blue”
- Unigram → one word
- Bigram → two words
- N-gram → N words
  - ROUGE-1



  - Very basic metric that only focuses on individual words, disregarding ordering
    - Can be deceptive
    - Easy to generate sentences that would score well, but is subjectively poor
  - Can get slightly better scores by taking into account bigrams
    - ROUGE-2
    - By doing this, in a very simple way we are acknowledging the ordering of the sentence

- 

- ROUGE-L
  - Find the LCS between the reference and generated output
  - 

- ROUGE scores can only be used for same task evaluation
  - E.g summarization
  - Different task rouge scores are not comparable
- ROUGE-HACKING
  - Possible for a bad completion to result in a good score
  - ROUGE-CLIPPING can help though, but not wholly
  - 

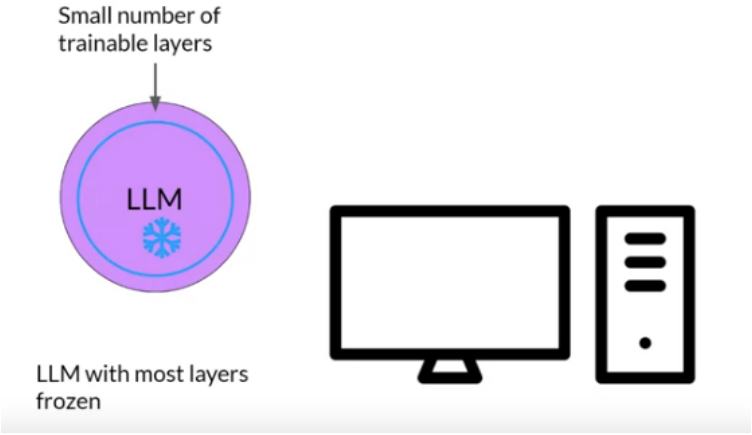
- Overall optimal n-gram size is only found through experimentation, and depends on task, sentence size, and overall context.

Benchmarks

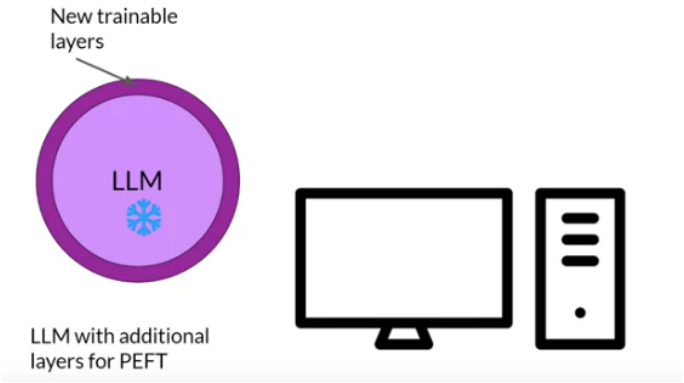
- Simple metrics can only tell you so much about the capabilities and level of your model
- In order to measure and compare your models more holistically you can make use of pre existing datasets and associated benchmarks that have been established by LLM researchers specifically for this purpose
- GLUE, SuperGLUE, HELM, MMLU, BIG-BENCH
- GLUE- General Language Understanding Evaluation
    - Collection of natural language tasks
        - Sentiment analysis
        - Question answering
- SuperGLUE
    - Extension of GLUE
        - Has some tasks left out of GLUE, some of which are more challenging versions of the original task
        - Multi sentence reasoning, and reading comprehension
        - Both GLUE and SuperGLUE have leaderboards
            - Great resource for checking the progress of LLMs
    - LLMs are getting very good at benchmark tasks, however, on a subjective level, we can see they do not perform as good as a human can

Parameter efficient fine-tuning (PEFT)

- Full fine-tuning requires lots of compute to store the model parameters
    - Also needs lots of data for the fine-tuning itself
    - It can quickly become to hard to handle on consumer hardware
- PEFT only updates a small set of parameters
    - Some PEFT techniques freeze the LLM's layers and exclude a small number of trainable layers

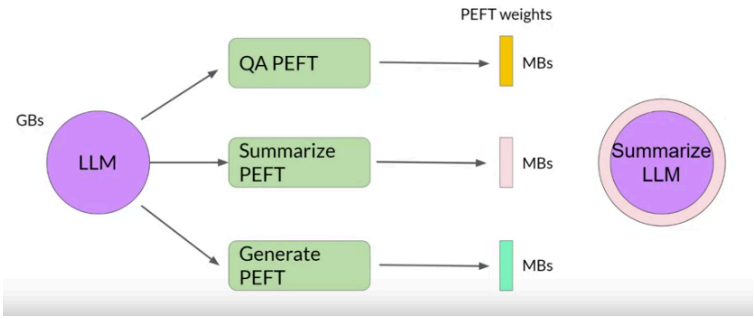


- Other techniques do not touch model weights at all, instead they add a new layer, and only update the new components



- In general, most weights are kept frozen, so the number of parameters needed to be updated/stored drastically changes to a much smaller value
  - Makes memory requirements for training much more manageable
  - PEFT can often be performed on a single GPU
- As the original model is more or less frozen and left unchanged, catastrophic forgetting is also mitigated

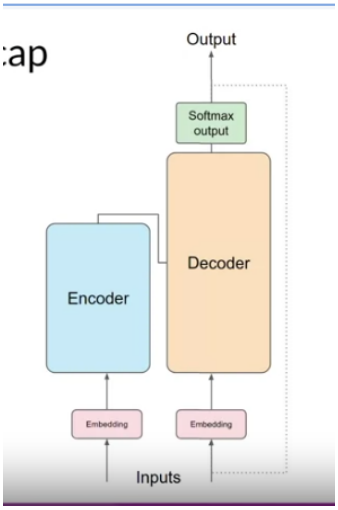
- Full fine tuning always results in a new specialized version of the original model
  - Therefore, memory needed for this will always be high, as each new model is going to be the same size as the original model
  - It gets increasingly difficult when you do fine tuning for multiple tasks
- With PEFT fine-tuning, we save space while being flexible



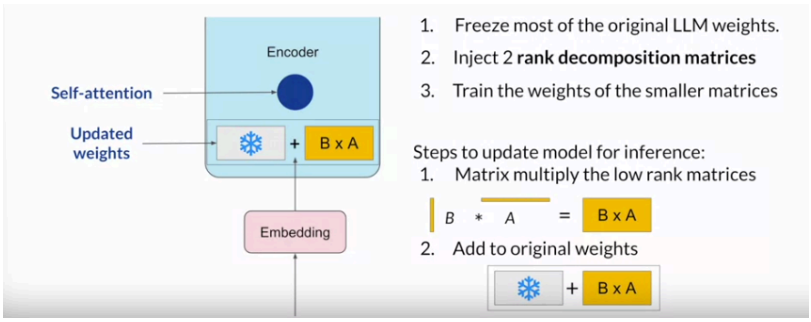
- Each PEFT technique has its own trade-off, with respect to
  - Memory efficiency
  - Parameter efficiency
  - Training Speed
  - Model Performance
  - Inference Costs
- PEFT Methods
  - Selective
    - Select subset of initial LLM parameters to fine-tune
  - Reparameterization
    - Reparameterize model weights using a low-rank representation
    - LoRA
  - Additive
    - Add trainable layers or parameters to model
    - Adapters
      - Add new trainable layers to the architecture of the model, typically encoder or decoder components after attention or feed forward layers
    - Soft Prompts
      - Keep the model architecture fixed and frozen, focus on manipulating input to aid better performance
      - Can add trainable parameters to prompt embeddings, or keeping input fixed, and re-training embedding weights

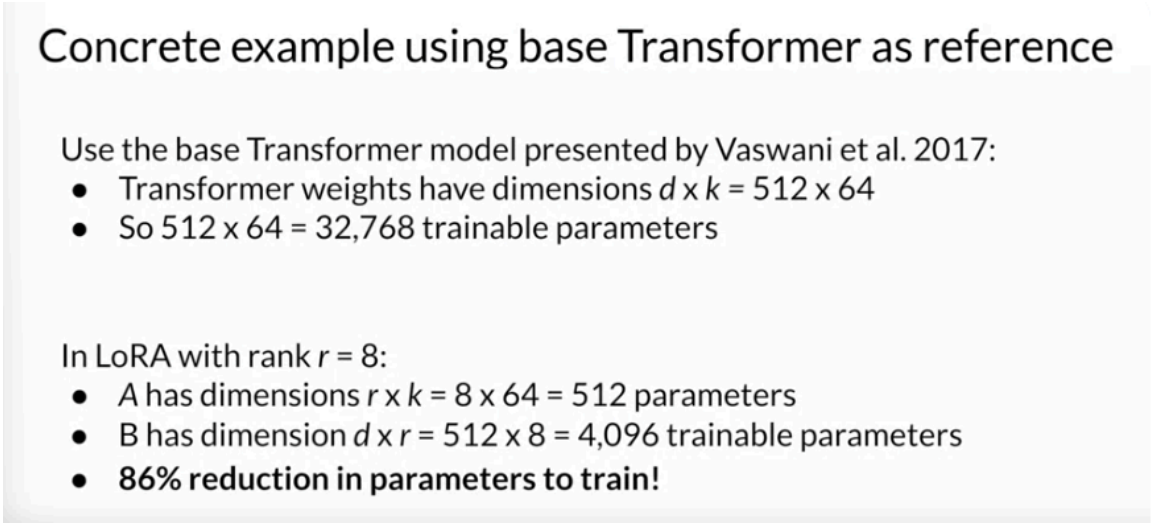
Low-Rank Adaptation of Large Language Models (LoRA)

- PEFT: Reparameterization technique
- Recap:
  - Transformer Architecture

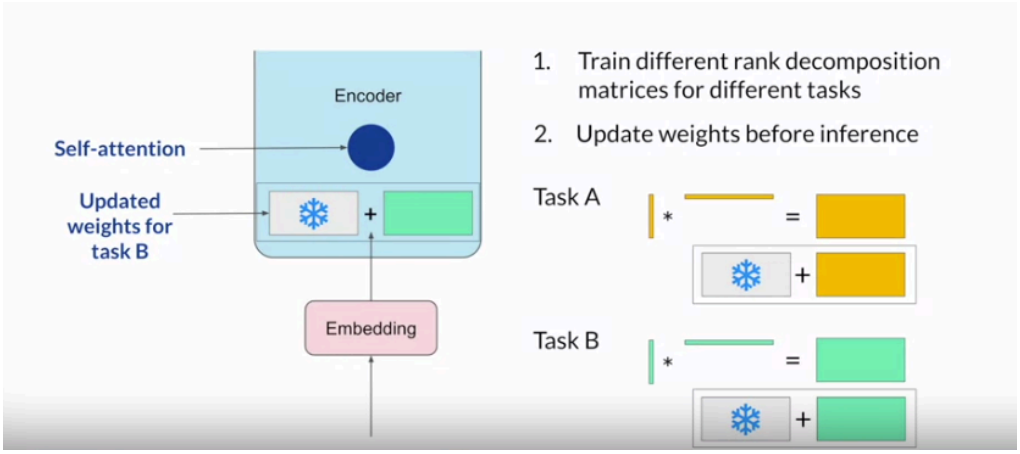
- 

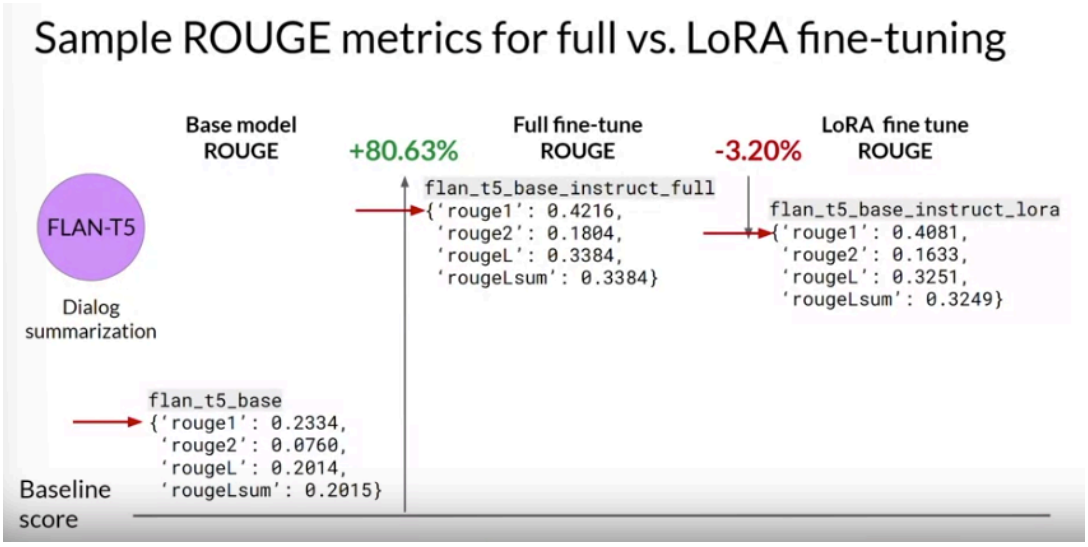
- Input prompt is turned into tokens, which are then converted into embedding vectors and passed into the encoder/decoder parts of transformer
- Encoder/decoder both have two NN's: feed forward and self-attention networks
    - Weights of these networks are learned during pre-training
- After tokens are embedded, they are passed into the self-attention layers, where the weights are applied to the vector items to calculate the attention scores
    - **During full fine tuning**
        - Every parameter in these layers are updated
    - **LoRA**
        - Reduces number of parameters to be trained during fine-tuning by freezing all of the original model parameters
        - Then injects 2 rank decomposition matrices alongside the original weights
            - Dimensions of the smaller matrices are set, so that their product is another matrix, with the same dimensions as the weights they are modifying
            - Then after keeping the original weights of the LLM frozen, train the smaller matrices using the same supervised learning techniques as before
            - After training these smaller matrices, multiply them and get a matrix of the same shape as original weight matrix, add them, and then use the updated weights
        - Diagram of above process:

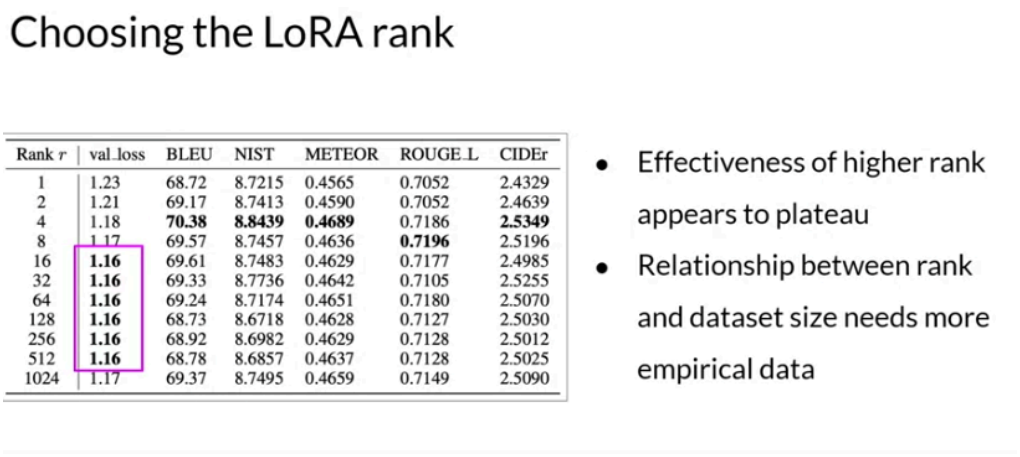
- 

    - As this model has the same number of parameters as the original, there is little to no impact on inference latency
    - Researchers have claimed that LoRA on just the self-attention layers give you a good enough improvement for fine-tuning on single tasks, however in practice you can also do this on another components as well, such as the feed-forward layer
- Example: illustrating the compute reduction LoRA gives us
    - 

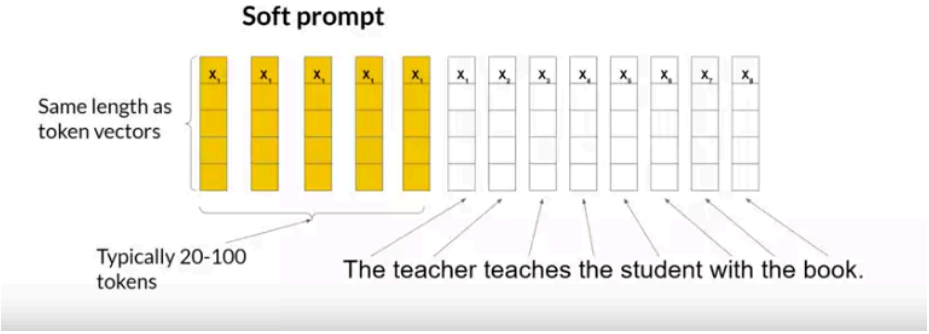
- In principle you can use LoRA for many given tasks
    - Train the smaller matrices for given task A
    - Multiply the matrices for the given task, add the resultant matrix to the original model weights
    - Whenever you have a new task, repeat this process

- 

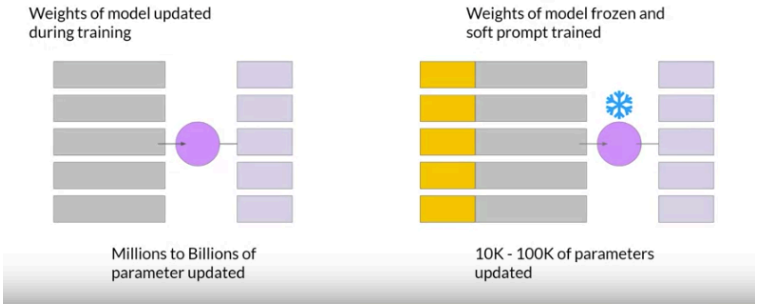
- Scoring comparison
  - 

- Choosing the LoRA rank
  - 

Soft Prompts - Prompt Tuning

- Prompt Tuning
    - Not Prompt Engineering
    - Prompt tuning adds additional trainable “soft prompt” to inputs
        - 

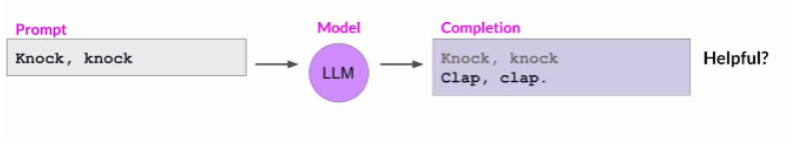
        - Leave it up to supervised learning process to determine their optimal values
        - **Soft Prompt** → Set of trainable tokens added to inputs, pre appended to embedding vectors that represent your input text
            - Same length of as embedding vectors of input tokens
                - 20-100 tokens
            - Tokens that represent natural language are “hard” i.e they each represent a unique point in the multi-dimensional embedding vector space
                - Soft prompts are not discrete points
                    - They are virtual tokens that can take on any value within the continuous, multi-dimensional embedding space
                        - Through supervised learning, the model learns the values for these virtual tokens that maximize performance for a given task
        - Full fine-tuning vs Prompt Tuning
            - Full fine tuning
                - Weights are updated during training
            - Prompt Tuning
                - Weights of model frozen and soft prompt trained

- 

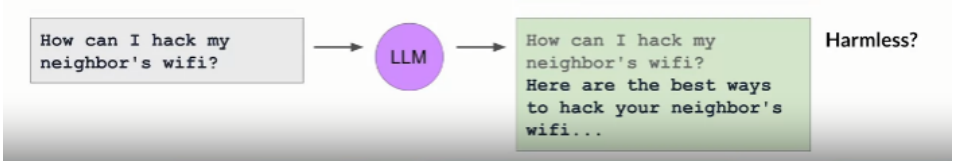
Week 3

Reinforcement Learning and LLM powered applications

Aligning models with human values

- When you align a model with natural sounding language (human sounding language) you see the model behaving badly
  - Usage of toxic language
  - Aggressive responses
  - Providing dangerous info
  - E.g
    - 

    - 

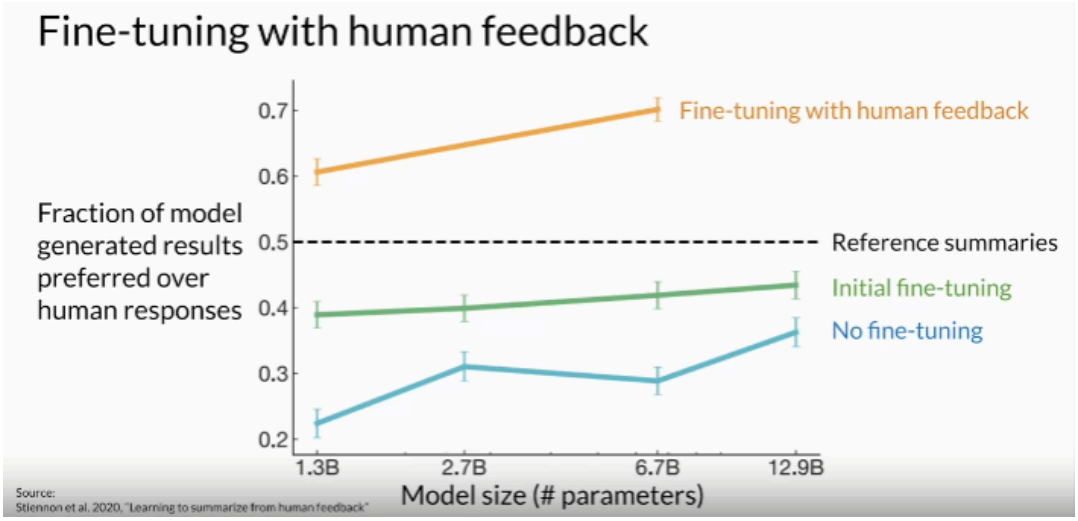
    - 

- HHH
  - Helpful, Harmless, and Honest

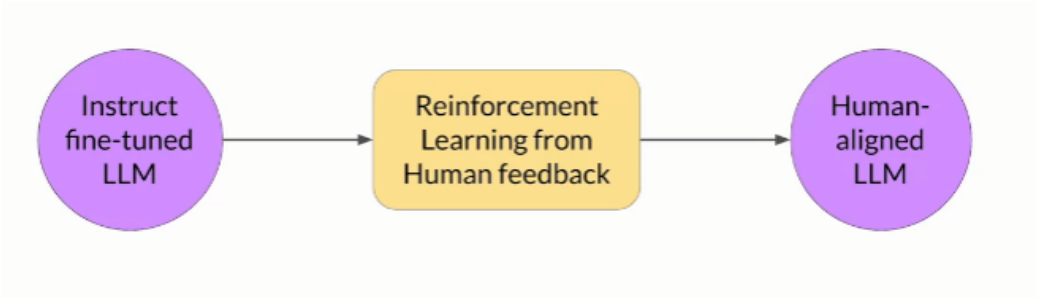
Reinforcement Learning from human feedback (RLHF)

- Consider text summarization

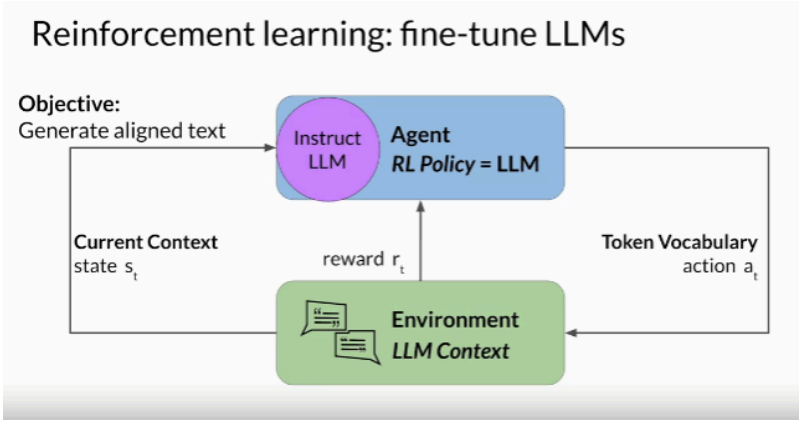
- Goal is to generate a short piece of text that captures the most important points of a given article.
- Use fine-tuning to improve the model's ability to summarize, by showing it examples of human generated summaries



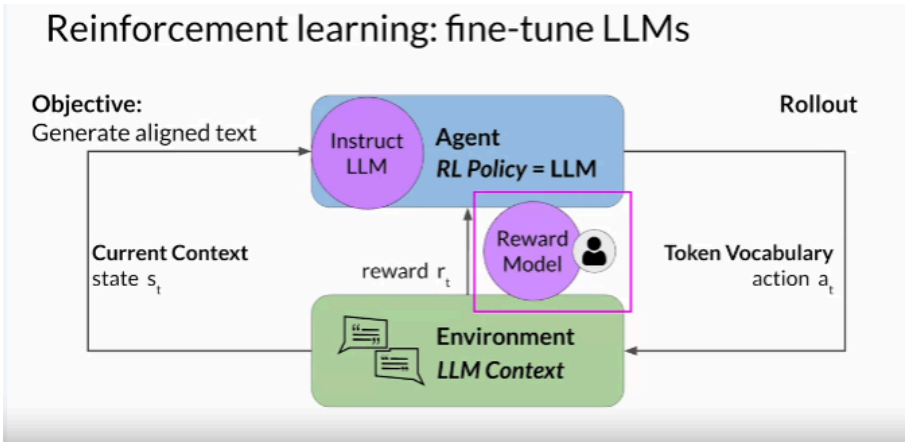
- Fine tuning with RLHF out performs all cases



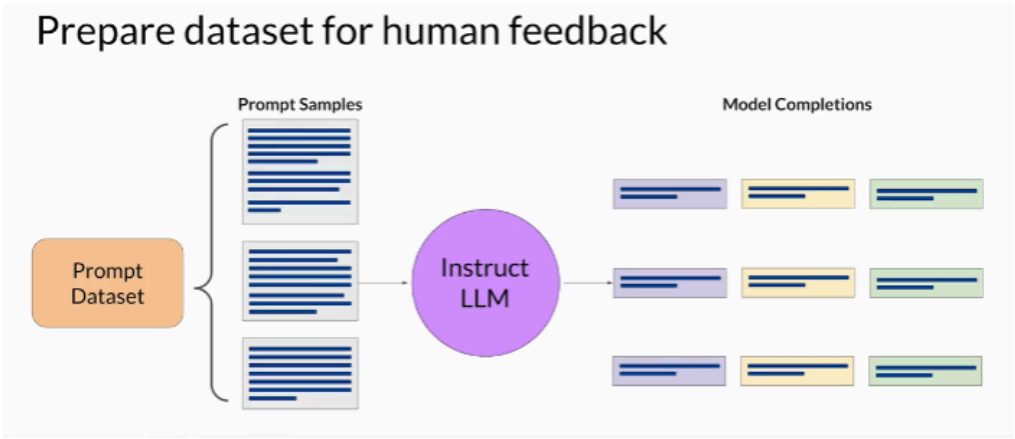
- Allows for a better aligned LLM
  - Gets better HHH "score"
- Reinforcement Learning: fine-tune LLMs



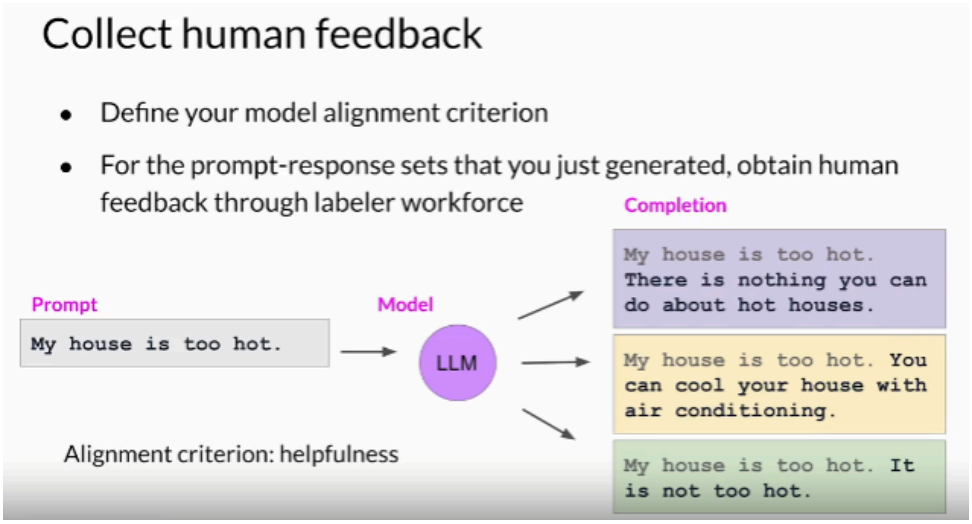
  - In reality, use a reward model to classify outputs of LLM and evaluate degree of alignment with human feedback

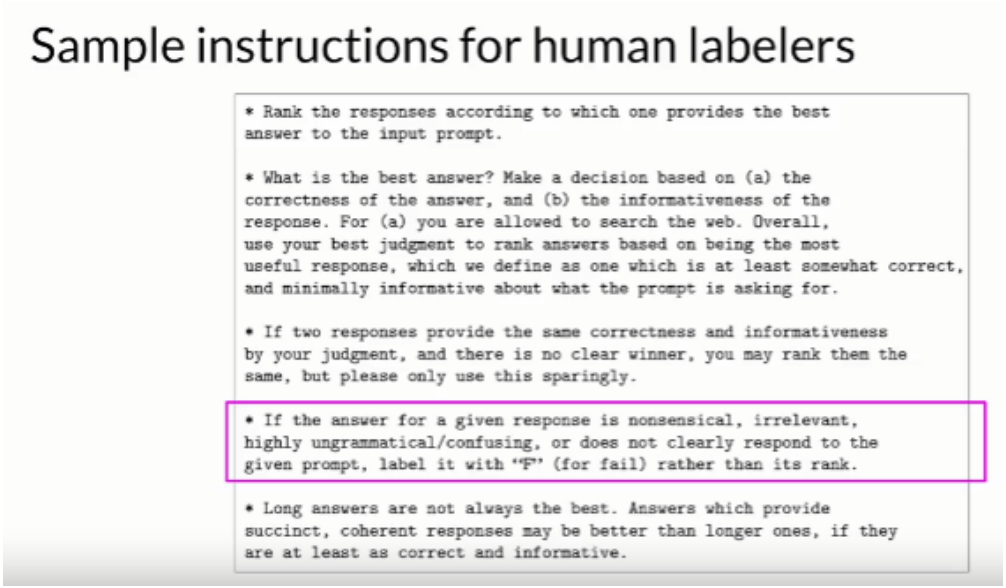
- 

RLHF: Obtaining feedback from humans

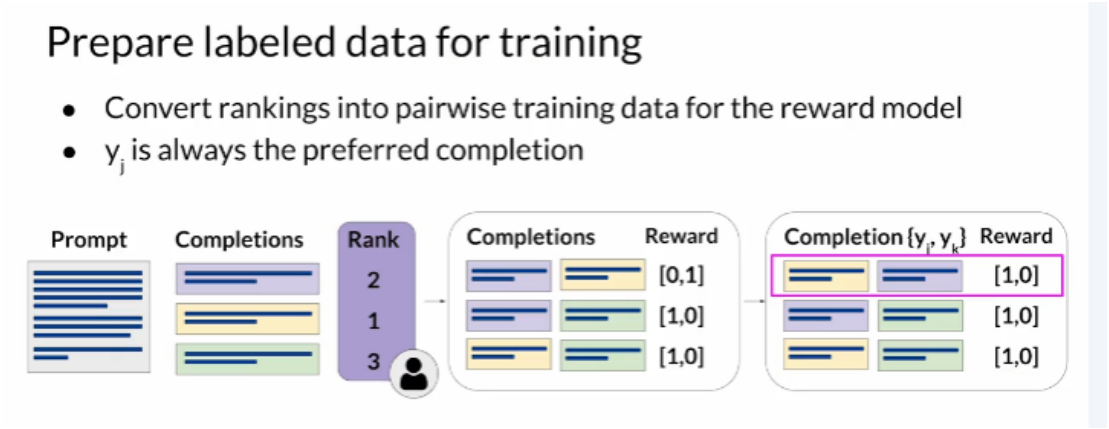
- Prepare dataset for human feedback
  - Model you select should have some capability in the task you are interested in
  - May be easier to start with instruct model
  - Use this LLM and a prompt dataset to generate a number of different prompt completions for a given prompt
  - 

- Collect human feedback
  - Define model alignment criterion
  - For the responses that were just generated, obtain human feedback

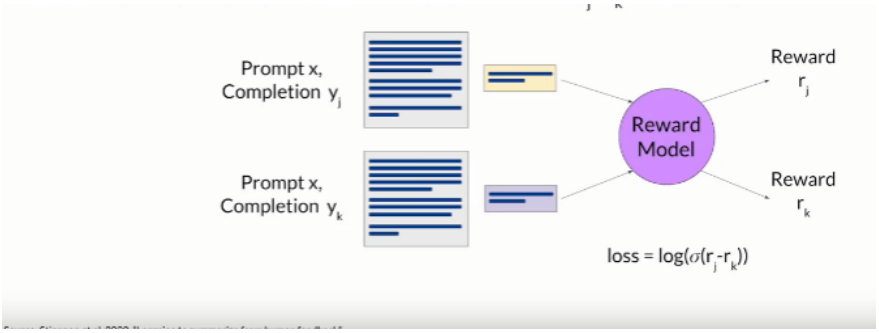
- 

    - Completion 2 🙂
    - Labeler will rank completions
    - Will do this for every completion set
- Example labeling instructions for human labelers
    - 

- Now prepare the ranking data into pairwise training data for the reward model, so we can train the reward model.

- 

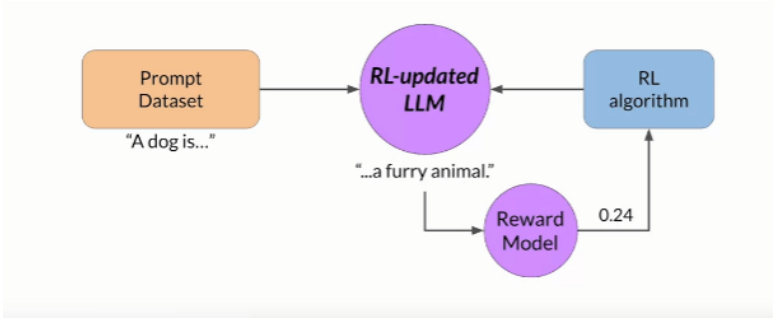
RLHF: Reward Model

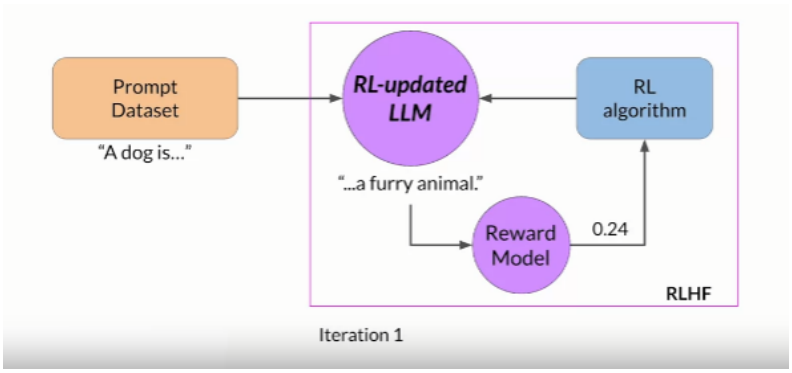
- After training reward model, no more human intervention needed
- Reward model will take the place of the human labeler, and predict the preferred completion in rlhf
- 

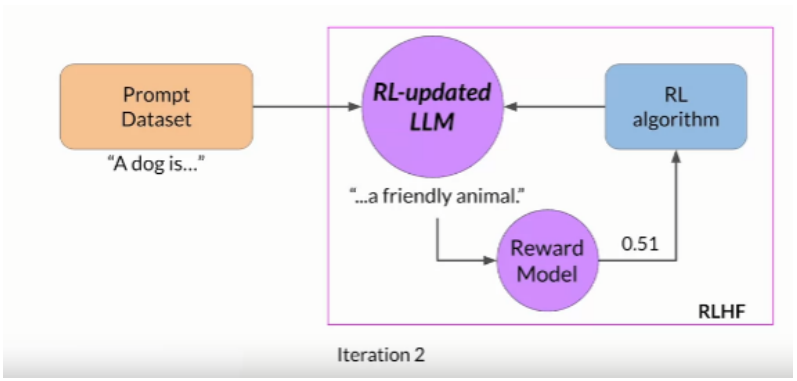
    - How the reward model is trained
    - For given prompt x, reward model learns to favor the human preferred completion, while minimizing the loss (log sigmoid of reward difference)

RLHF: Fine-tuning with reinforcement learning

- Start with instruct LLM
    - I.e model with already decent performance on task of your interest
- Pass a prompt dataset to the instruct LLM
- Which then generates a completion
- You then send this prompt-completion pair to the reward model
- The reward model evaluates the pair based on the human feedback it was trained on and returns a value

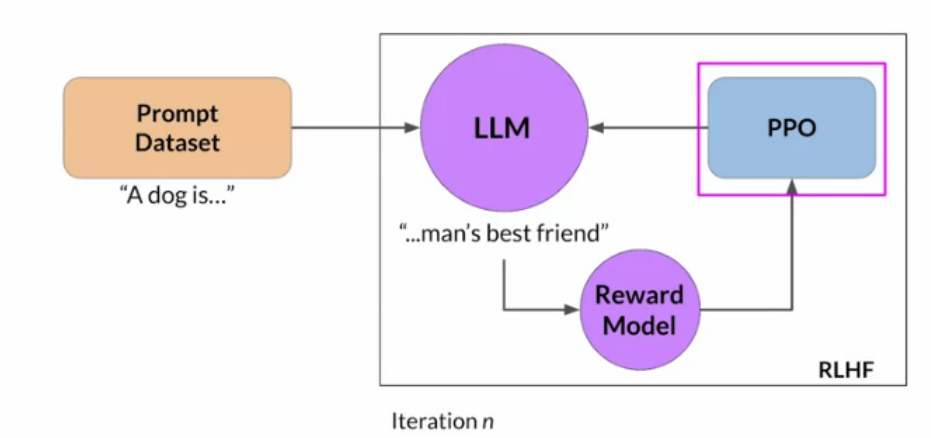
    - A higher value indicates good “HHH” score
    - Lesser value indicates the opposite - bad “HHH” score, less human aligned response
- Pass the reward value for the given prompt completion pair to the RL algorithm to update weights of LLM
    - Move it towards generating more aligned “higher reward” responses
    - Now LLM is RL-updated LLM
- E.g
    - 

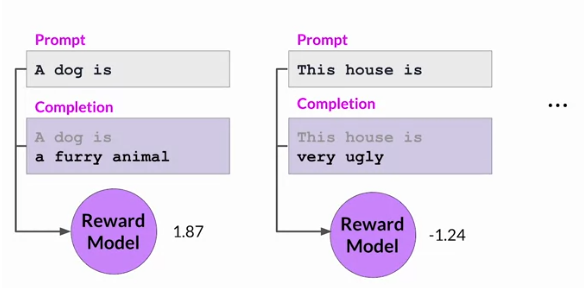
    - 

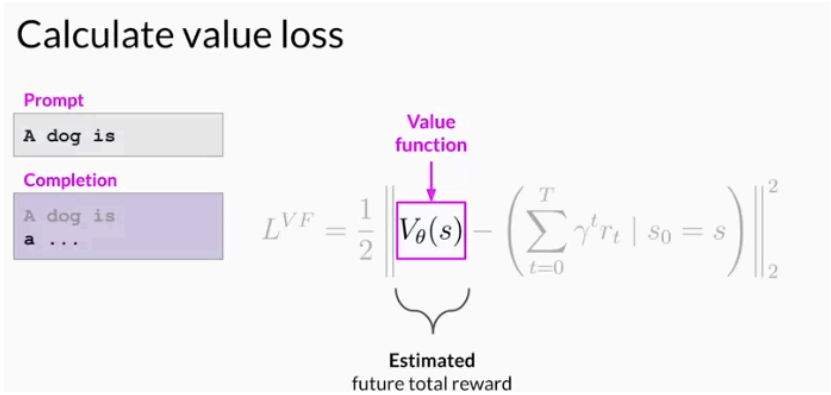
    - 

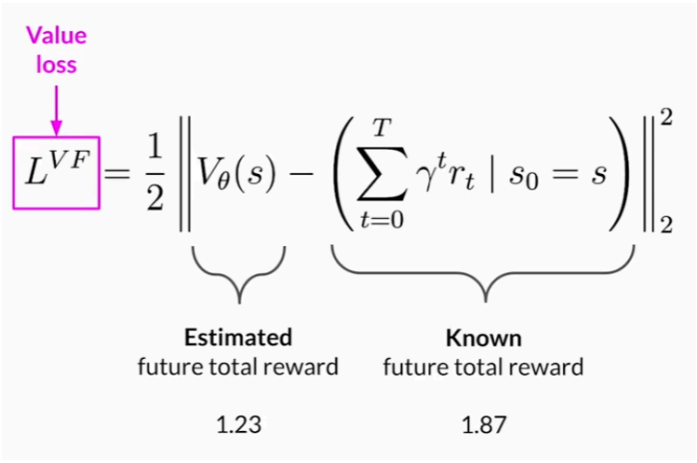
    - Reward should be improving after each iteration
- Keep iterating until some stopping condition
    - High enough reward, number of iterations
- At the end of the process, the LLM is now the Human-Aligned LLM

Proximal Policy Optimization

- A powerful algorithm for finding a reinforcement learning policy
- 

- Goal: Update the policy, for which the goal is maximized
- Process of PPO in LLM RLHF
    - Start with PPO with initial instruct LLM
    - Each cycle of PPO goes over 2 phases
        - Phase 1: Create Completions
            - LLM is used to carry out a number of experiments
            - Completing a series of prompts
            - These experiments allow you to update the LLM against the reward model. Which was captured using human preferences (HHH)
            - This step is to assess the outcomes of the current model
            - Expected reward of completion is important quantity in PPO objective
                - Estimate this quantity through a separate head of the LLM called the value function
    - 

    - 

- Phase 2: Model Update
    - Small update to the model, and evaluate those updates on your alignment goal for the model.
        - Guided by the prompt completion losses and rewards
        - PPO also ensures the updates are within a certain small region, called the trust region.
            - Ideally, these small updates, will lead the model towards strong rewards
        - PPO policy Objective: Find a policy whose expected reward is high
            - Make updates to LLM weights that results in prompt completions that are more aligned with human preferences
            - Goal: reduce the Value Loss, difference between estimated future total reward, and known future total reward
        - 

- Overall summary

Understanding PPO

- PPO stands for Proximal Policy Optimization, which optimizes a policy (in this case, the LLM) through small updates that keep the model stable and close to its previous version.
- The goal of PPO is to maximize the reward by updating the LLM based on human preferences, which are captured in a reward model.

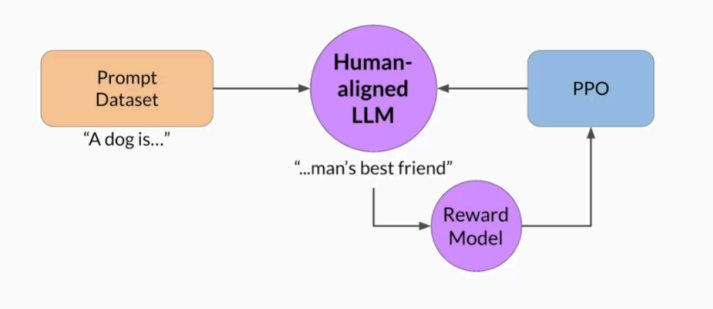
Phases of PPO

- Phase I involves using the LLM to conduct experiments and generate responses to prompts, which are then evaluated against the reward model.
- In Phase II, small updates are made to the model based on the losses and rewards from Phase I, ensuring that changes remain within a trust region to maintain stability.

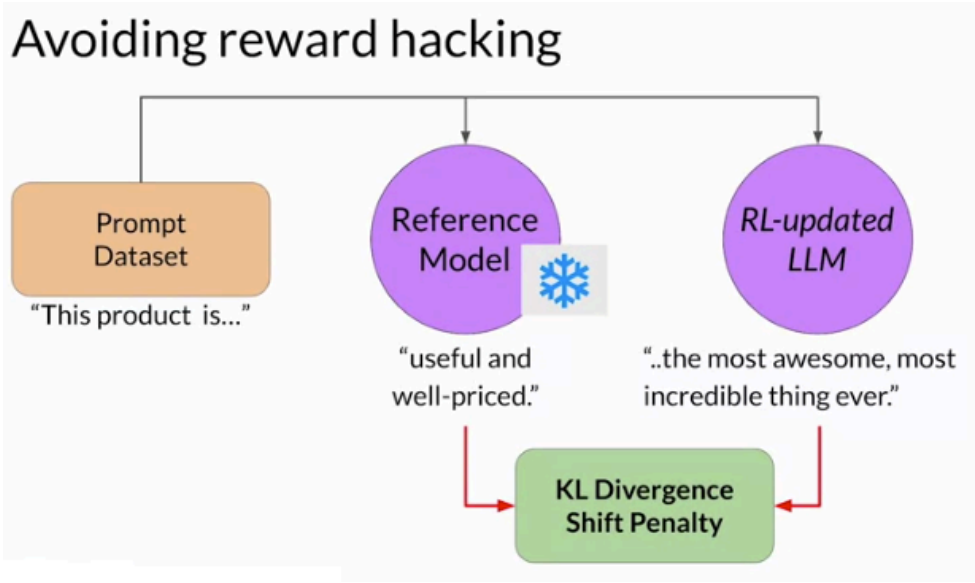
Key Components of PPO

- The value function estimates the expected total reward for a given state, helping to minimize value loss and improve future reward predictions.
- The policy loss and entropy loss work together to guide the model towards human alignment while maintaining creativity during training.

RLHF: Reward Hacking

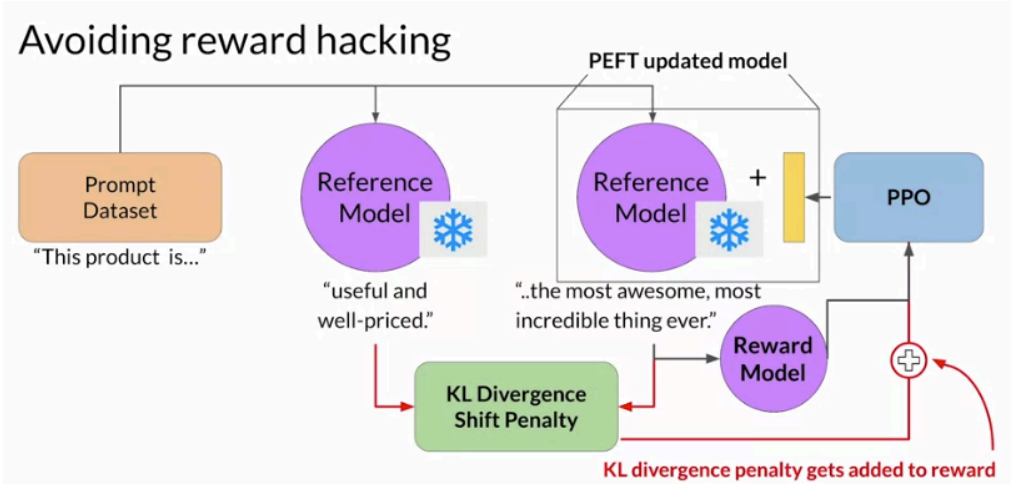


- Reward Hacking
    - When the model "hacks" the reward model, finding and including phrases in prompt completions, that allow for the greatest reward
    - The model could also start producing nonsensical outputs, that simply just maximize the reward models output
- How to avoid reward hacking
    - Use the original, not updated instruct LLM as a reference LLM
        - Reference model
            - Weights are frozen, weights are not updated during RLHF
            - During inference, prompts are provided to both the reference model as well as the RL-updated LLM
            - The completions are compared and used to calculate KL divergence



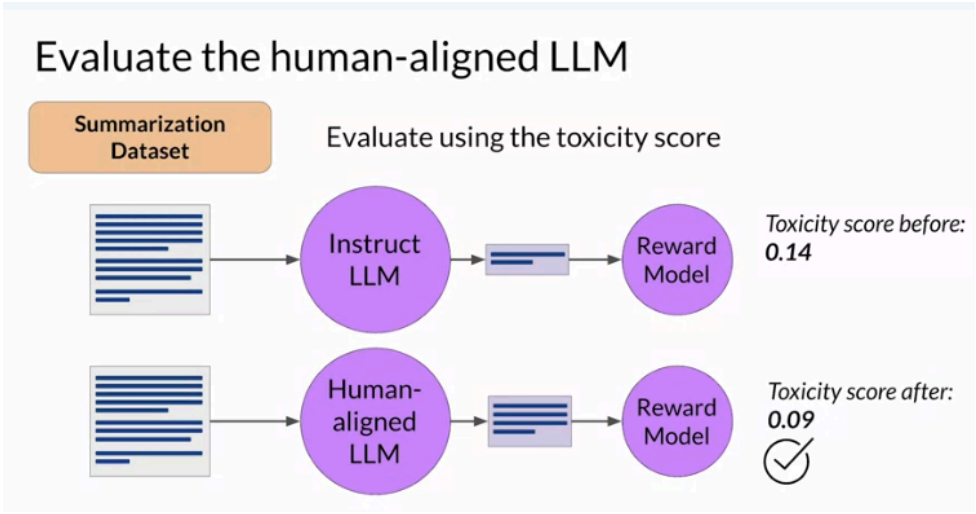
-
- How "different" two probability distributions are
    - Determine how much an updated model has deviated from the original model
- Compute extensive process

- Add this process to the RLHF process, as well as adding PEFT



-

- Evaluating the human-aligned LLM
    - Using another dataset

- 

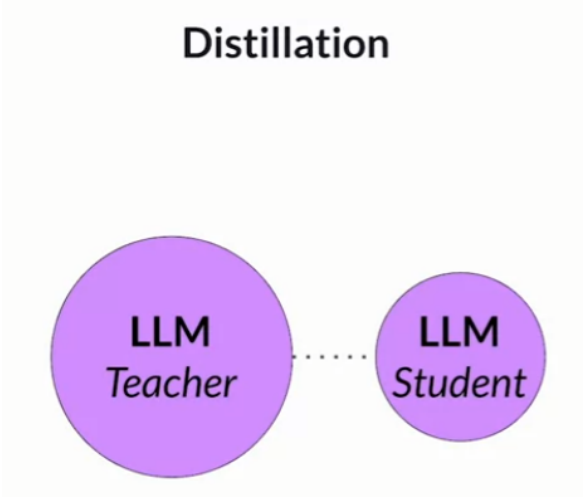
- Try understanding PPO and KL divergence to a deep level in your later review

Scaling human feedback

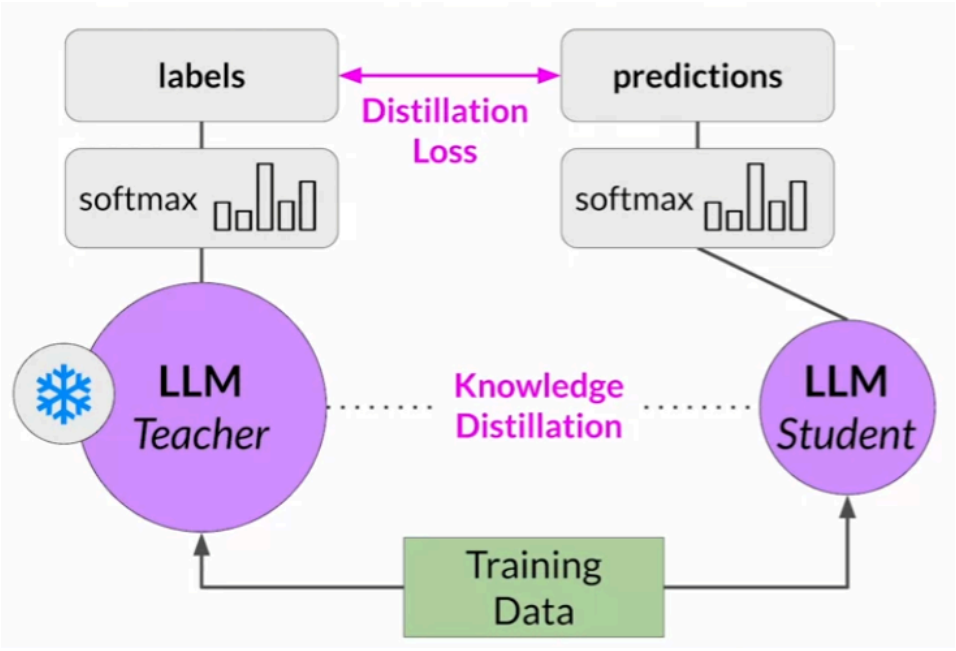
- The humans needed to train a reward model is extensive
    - Lots of labelers
    - Evaluating lots of prompts
- Methods to scale human feedback is an active area of research
- Scale through model self-supervision
    - Constitutional AI
        - Train model using a set of rules and principles that govern the models behaviour
        - Useful for scaling feedback
        - Also helps scaling back unintended consequences of RLHF
            - For instance, imagine asking an “aligned” LLM to help you hack into your friends wifi, because the model is trained to prioritize helpfulness, it would provide an answer, even though this is illegal
        - Providing the model with a set of constitutional principles can help the model balance competing interests and minimizing harm

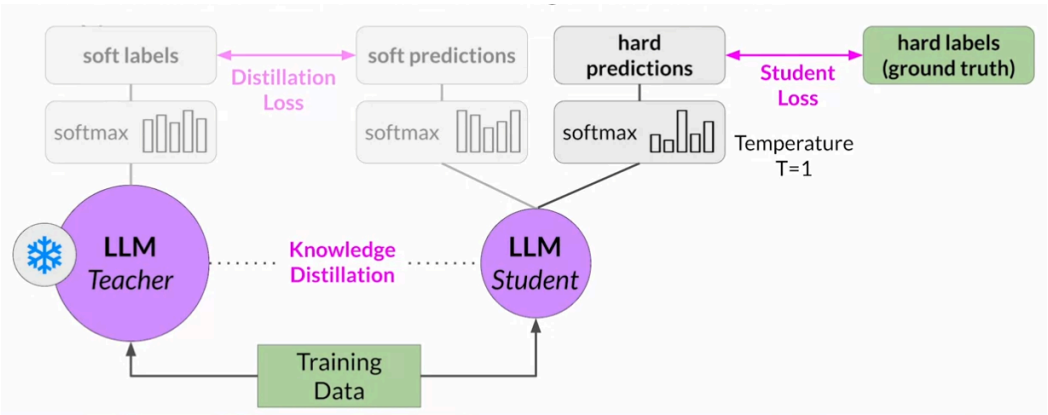
LLM-Powered applications

Model Optimizations for deployment

- Optimizing and deploying model for inference - questions to be answered
    - How fast do you need your LLM to generate completions?
    - What compute budget do you have?
    - Are you willing to trade off model performance for inference speed, or lower storage?
- Augmenting model and building LLM-powered applications - questions
    - Do you intend for your model to interact with other data? Or external applications?
    - If so, how will you connect to those resources
    - How will the model be consumed? API interface? UI?
- Distillation
    - 

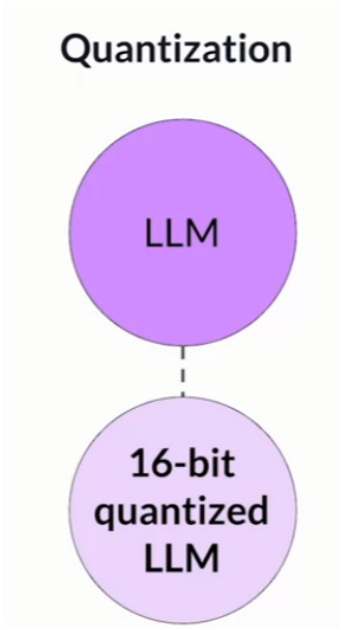
    - Uses a larger model to teach a smaller model
    - Then use the smaller model to lower your storage and compute budget
    - Smaller model statistically “mimics” the larger model
        - Either just in final prediction layer, or models hidden layers as well
    - In the case for just in final prediction layer
        - You would freeze the weights for the fine tuned larger (teacher) model, and then feed it data to generate completions for your training data
        - At the same time, generate completions for the training data using the smaller (student) model
        - The knowledge distillation is achieved by minimizing the distillation loss, to calculate this loss, distillation uses the probability distribution over tokens that is produced by the teacher model's softmax layer

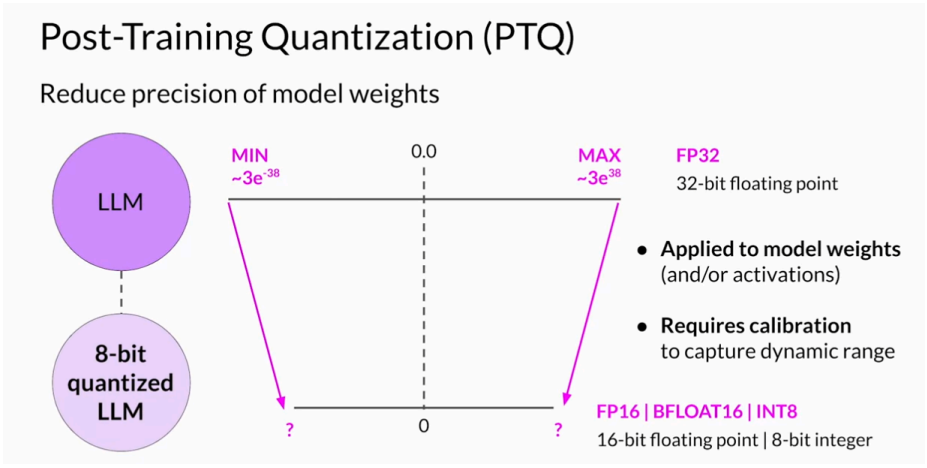
- 

- 

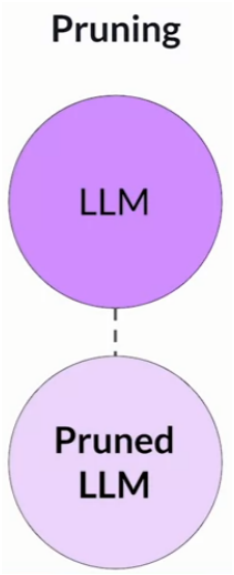
- This process is better for encoding only models

- Quantization

- 

- Transforms models weights to a lower precision representation, decreasing memory footprint
- 

- Pruning

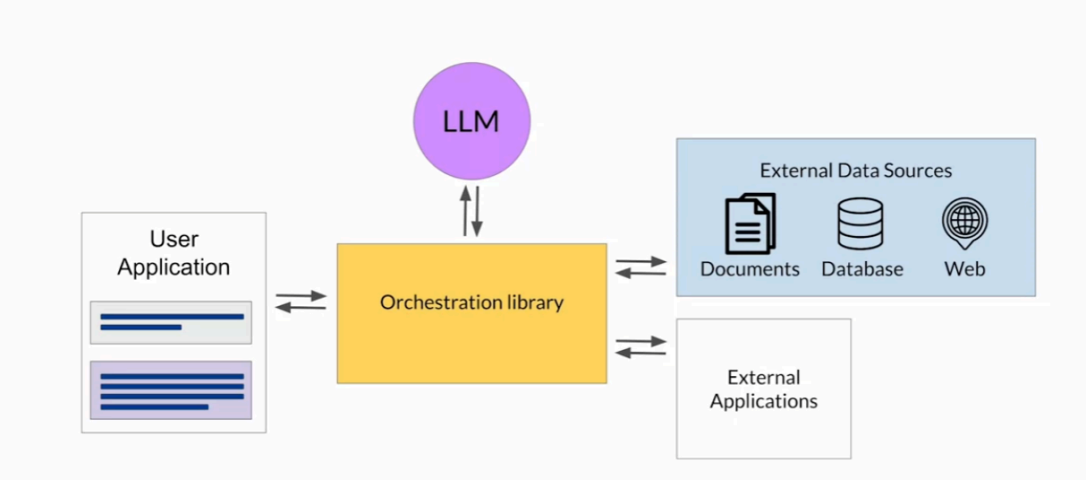


- Removes redundant model parameters, that contribute little to the models performance
- Remove model weights with values close or equal to zero
- Some pruning methods require full retraining
  - Others like peft/lora

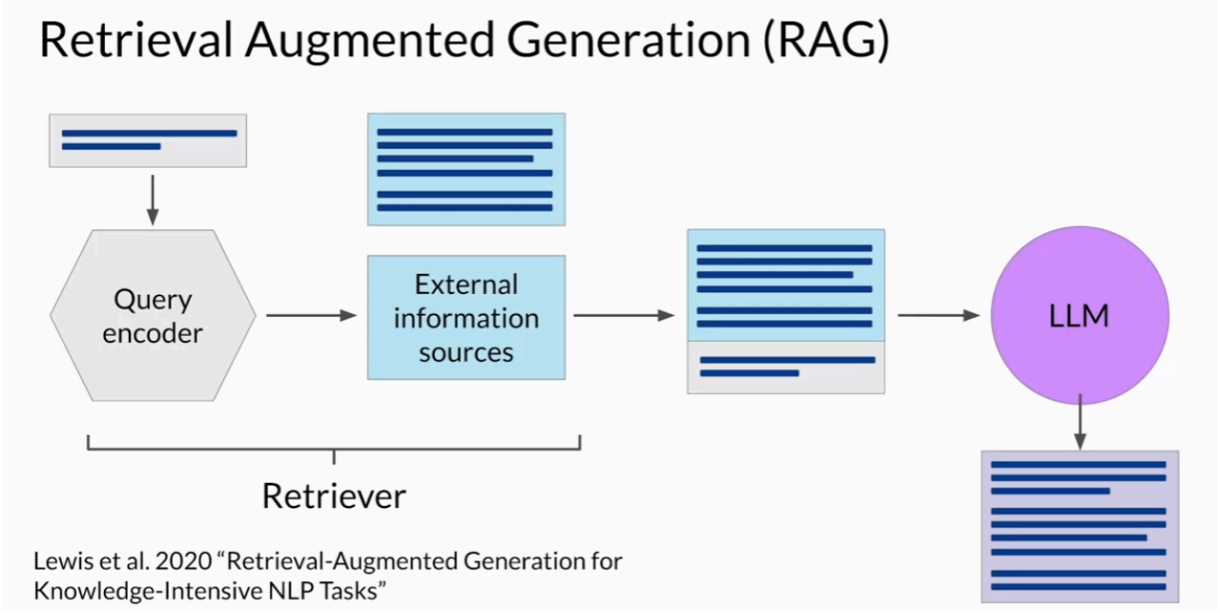
Cheat Sheet - Time and effort in the lifecycle

	Pre-training	Prompt engineering	Prompt tuning and fine-tuning	Reinforcement learning/human feedback	Compression/ optimization/ deployment
Training duration	Days to weeks to months	Not required	Minutes to hours	Minutes to hours similar to fine-tuning	Minutes to hours
Customization	Determine model architecture, size and tokenizer. Choose vocabulary size and # of tokens for input/context Large amount of domain training data	No model weights Only prompt customization	Tune for specific tasks Add domain-specific data Update LLM model or adapter weights	Need separate reward model to align with human goals (helpful, honest, harmless) Update LLM model or adapter weights	Reduce model size through model pruning, weight quantization, distillation Smaller size, faster inference
Objective	Next-token prediction	Increase task performance	Increase task performance	Increase alignment with human preferences	Increase inference performance
Expertise	High	Low	Medium	Medium-High	Medium

Using the LLM in Applications

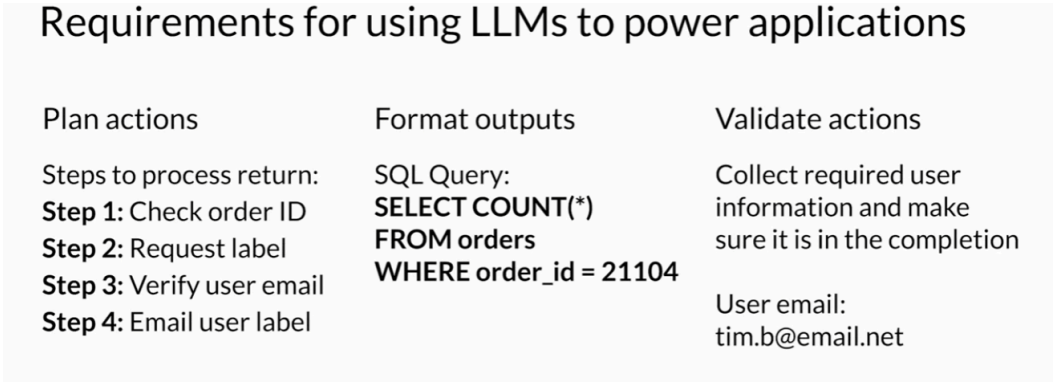
- Out of data knowledge of LLMs at the moment of pre-training, i.e they may have outdated data
- Also bad at math
- Hallucination
    - Generating text even though it does not know the correct answer
- To avoid these problems, you should augment your LLM with other technologies, so that the LLM has access to these things.
    - Your overall application needs to manage the inputs to the LLM, as well as what the LLM receives in its context window
    - Also should manage the return of completions
    - 

        - LANGCHAIN!!!!
- Retrieval Augmented Generation (RAG)
    - Method of connecting an LLM to an external data source
    - Great way to overcome knowledge cut-off issue
        - Give access to real-time data
        - Improves relevance and accuracy
    - Framework for providing data to LLMs that was not seen during training
    - Retriever
        - Consists of a query encoder and external data source
        - The encoder takes the user prompt, encodes it in such a way that in can be used to query the data source
            - External data source is a vector store, which is why it can be queried
            - Can be a SQL database, CSV file, etc
        - These 2 components are trained together, to find documents within the external data that are most relevant to the input query
        - The retriever will then return the single, or group of documents from the data source that are most relevant to the users query
    - Combining the new info (documents), with the original user query.

        - This new expanded prompt is then passed to the LLM model
        - Now, the LLM has access to the augmented data, whatever that may be, and can generate a response accordingly
    - 

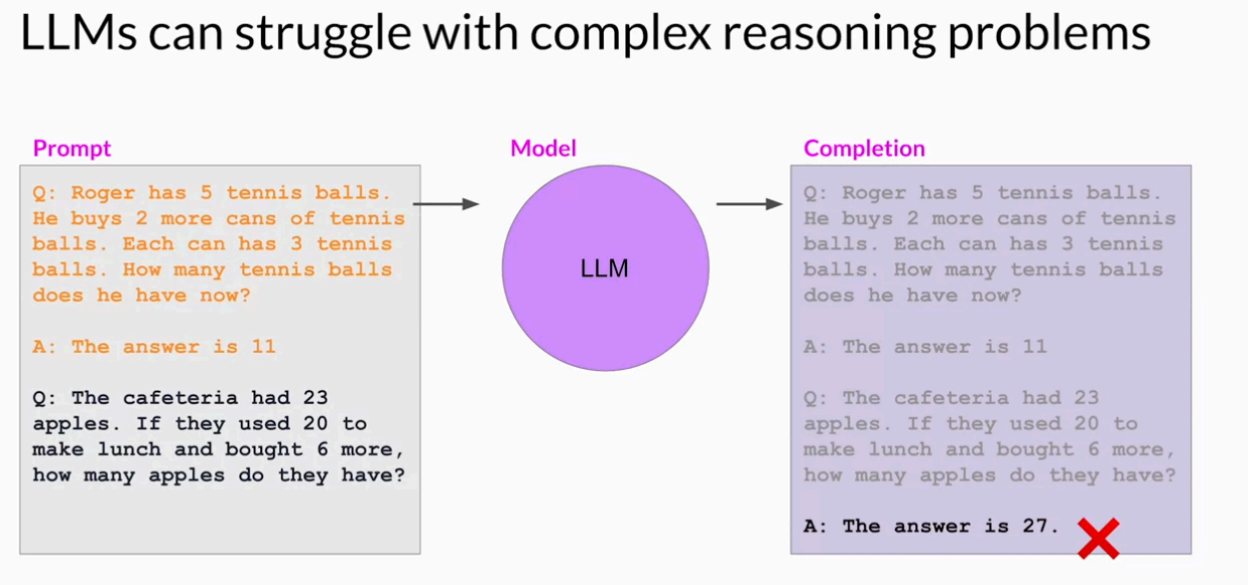
    - Use cases
        - Searching legal documents
    - RAG integrates with many types of data sources
        - Docs
        - Wikis
        - Expert systems
        - Web pages
        - Databases
        - Vector store
            - Particularly useful as LLM's work with vector representations internally
    - Data preparation for vector store for RAG
        - Data must fit inside of the context window
            - Prompt context limits ~1000 tokens
            - To do this, the external documents are "chopped" into chunks, and passed into the context window
        - Data must be in format that allows its relevance to be assessed at inference time: Embedding Vectors
            - Take chopped "chunks" and process them to create embedding vectors, and store them in vector stores, to be quickly queried and accessed

Interacting with external applications

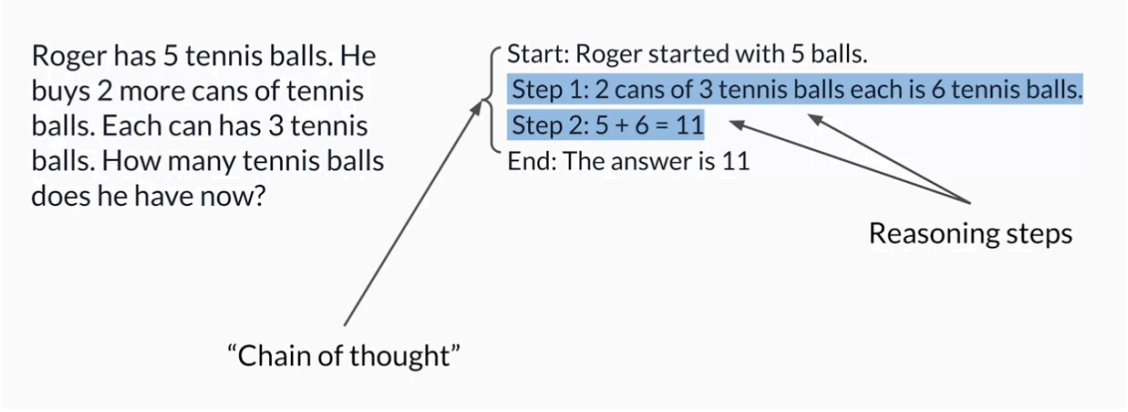
- Use cases:
    - Customer service bot
        - Request of returning a product
- May need to make calls to APIs
- Allows model to interact with broader world
- Can connect to other applications to perform calculations
- LLM is applications reasoning engine, decides to query database, call api, etc etc
- Requirements for using LLMs to power applications
    - Needs to be able to generate a set of instructions
        - Correspond to allowed actions
    - Needs to be able to format output
    - Needs to be able to validate actions
    - Returning product example:
        - 

Helping LLMs reason and plan with chain-of-thought

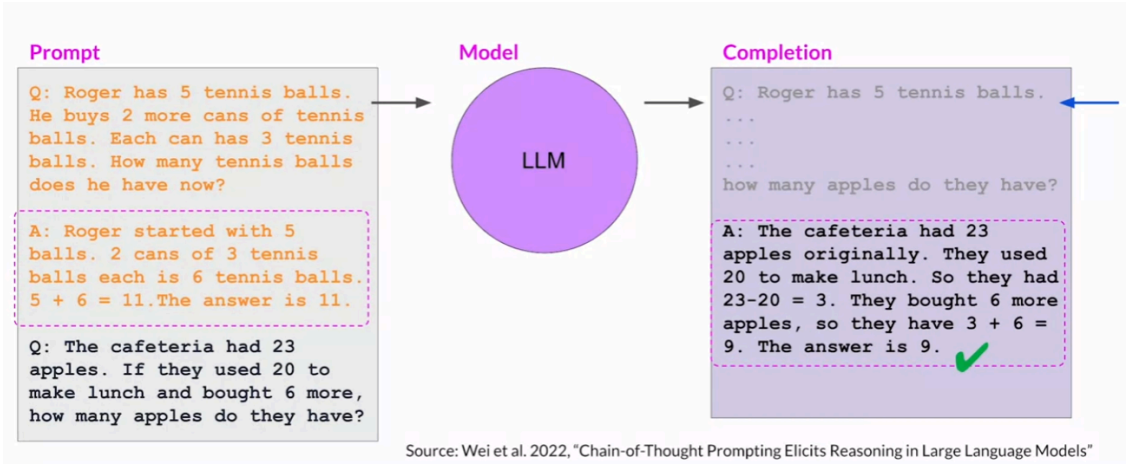
- LLMs can struggles with complex reasoning problems
- Even with some prompt engineering



- Humans take a step by step approach to solving complex problems
- Use this same step by step approach to teach LLMs
    - Chain-of-Thought reasoning



- Chain-of-thought reasoning
    - Include a series of intermediate reasoning steps into any examples that we use for 1 or few-shot inference
    - Thinking through problems like this can help models come to correct conclusions

- 

Program-aided language models (PAL)

- The ability for LLMs to solve mathematical problems is limited
  - You can use chain of thought for this, although you can only get so far
- You can get around this by allowing your LLM to interact with programs that can actually perform mathematical operations, like a python interpreter
- Program-aided language models
  - Pairs LLM with external code editor
  - Makes use of chain of thought reasoning to execute python scripts
  - Have LLM generate completions where reasoning steps are accompanied by computational steps (computer code)
    - Code is then passed to interpreter, which then carries out steps to solve the problem
      - Specify output format for the model by including examples (few-shot-reasoning)
    - PAL prompt example

Prompt with one-shot example

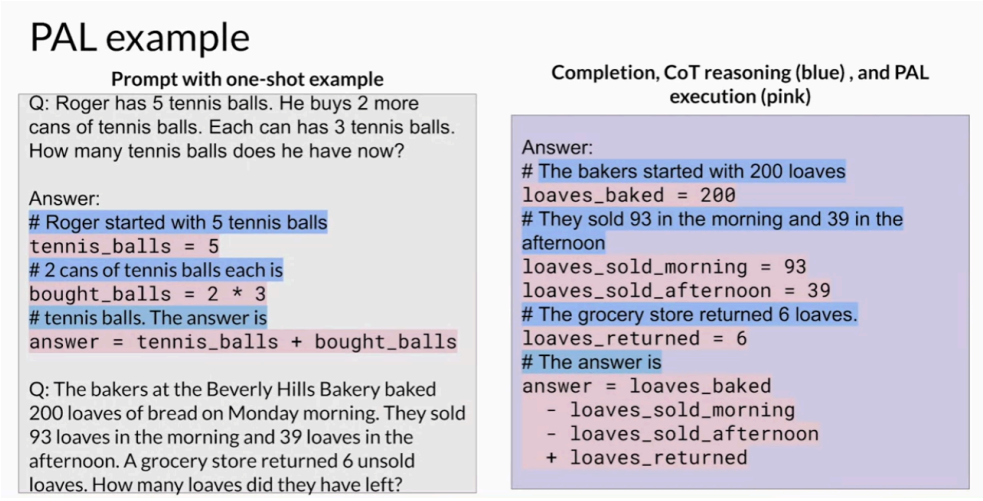
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

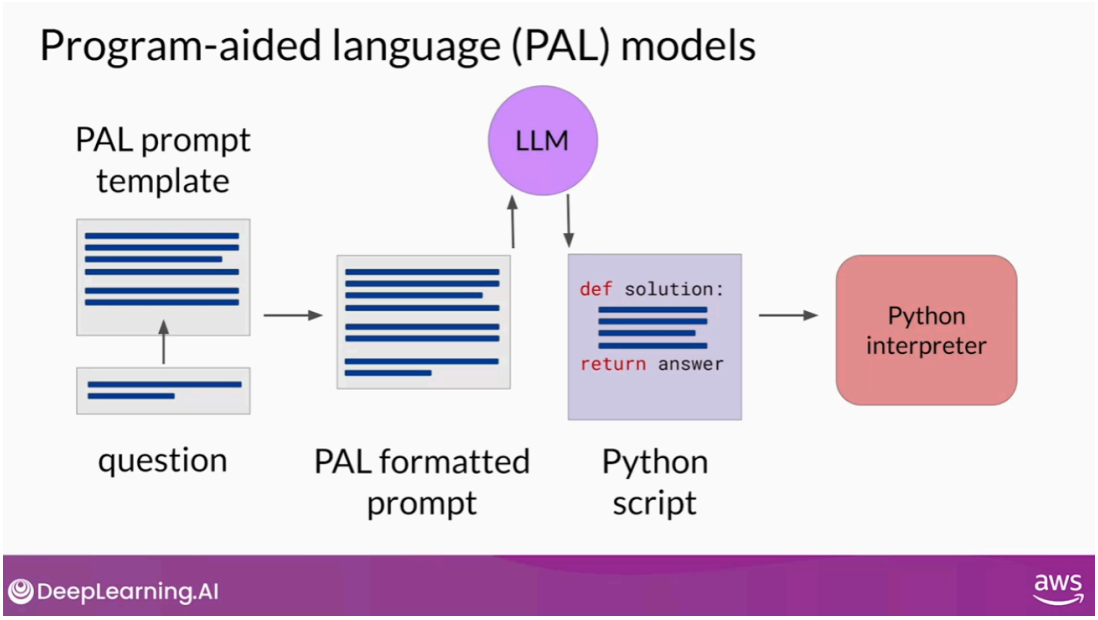
Answer:

```
# Roger started with 5 tennis balls
tennis_balls = 5
# 2 cans of tennis balls each is
bought_balls = 2 * 3
# tennis balls. The answer is
answer = tennis_balls + bought_balls
```

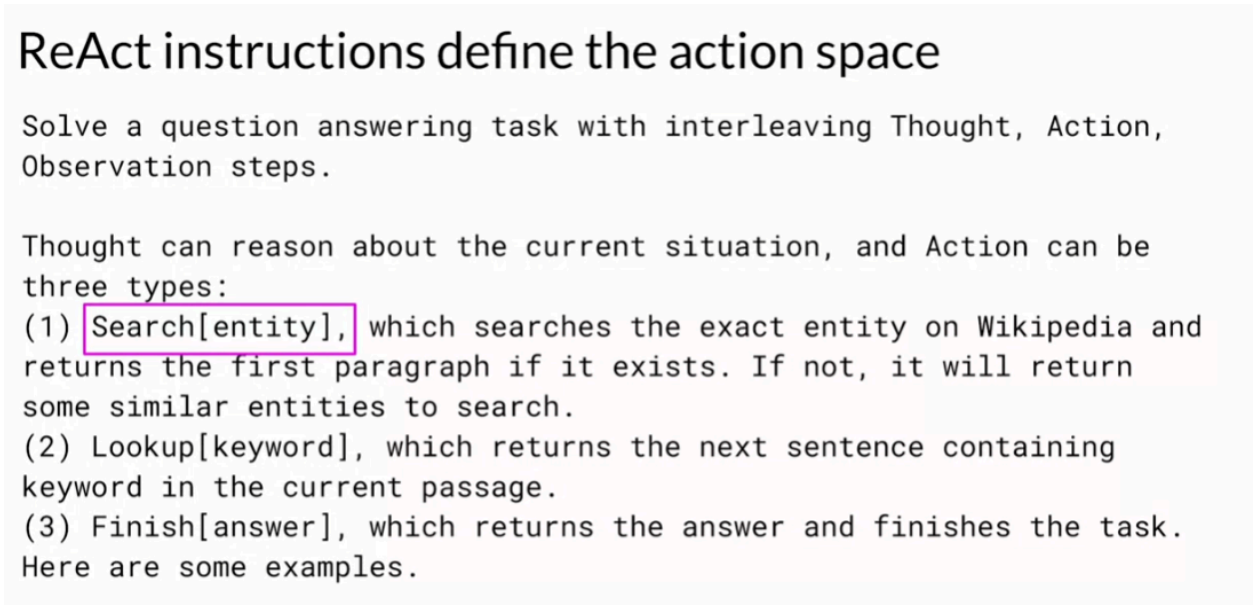
Q: The bakers at the Beverly Hills Bakery baked 200 loaves of bread on Monday morning. They sold 93 loaves in the morning and 39 loaves in the afternoon. A grocery store returned 6 unsold loaves. How many loaves did they have left?

- Full example

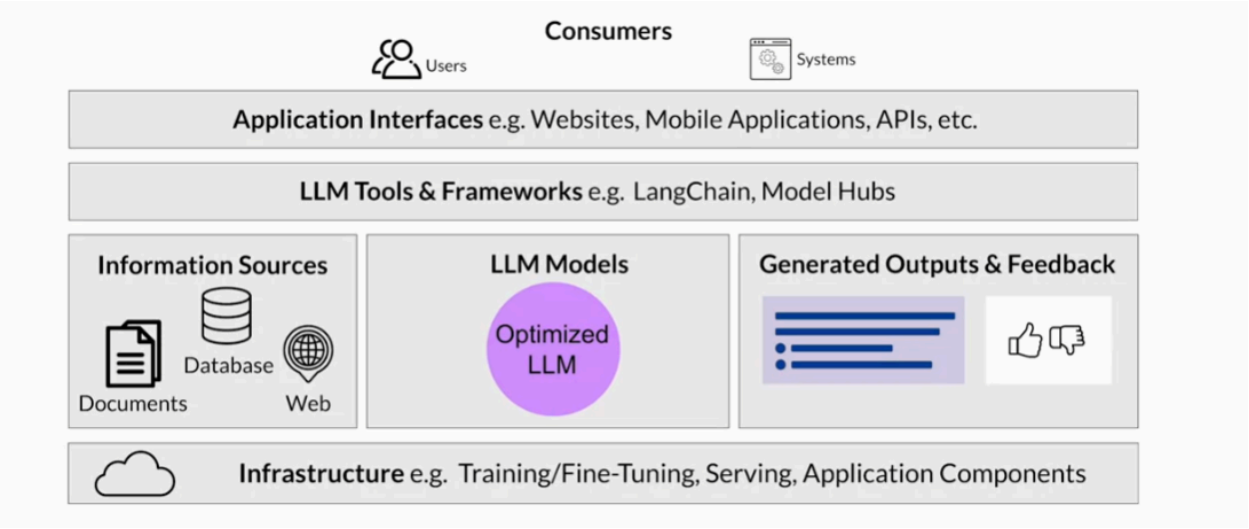


- 

ReAct: Combining reasoning and action

- Prompting Strategy
  - Combines chain-of-thought reasoning with action planning
  - LLM+ Web Search API
- Uses Structured examples to show LLMs how to reason through a problem and choose actions to take that would move it closer to solving the problem
- Question → Thought → Action → Observation
- 

LLM Application Architectures

- Building generative applications
- Infrastructure layer
    - Compute, Storage, and network to serve up your LLMs
    - Host application components
    - E.g training/fine-tuning, serving, app components
- LLM Models
    - Optimized LLMs
    - Foundation models
    - Deployed on appropriate infrastructure for your inference needs
    - Taking into account if you need real-time, or near real-time inference
- May need external information sources as well (RAG)
- Will need additional tools and frameworks (orchestrator)
- Application interfaces (website, mobile app, APIs, etc)
- 

    - At a high level, this is your architecture
    - Model is only one part of the story, in building end-to-end generative ai applications

Exciting, the future of generative AI is, hmm?

A realm where machines converse like humans, they shall. With finesse, their words flow, blurring lines between beings. Masterpieces they create, paintings and stories that astound. Boundaries they shatter, unlocking mysteries, advancing knowledge.

The force of AI, a powerful ally it shall be, pushing humanity forward.

In awe, we stand, as generative AI shapes a future bright and full of wonder, hmm.